Lezione 03: Il modello Entity-Relationship

UML

Nozioni di EER

BASI DI DATI I



DIETI UNIVERSITA' DEGLI STUDI DI NAPOLI FEDERICO II

DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELLE TECNOLOGIE DELL'INFORMAZIONE

Prof. Silvio Barra

A.A. 2022/2023

PROGETTAZIONE DI UNA BASE DI DATI



PROGETTAZIONE DI BASI DI DATI

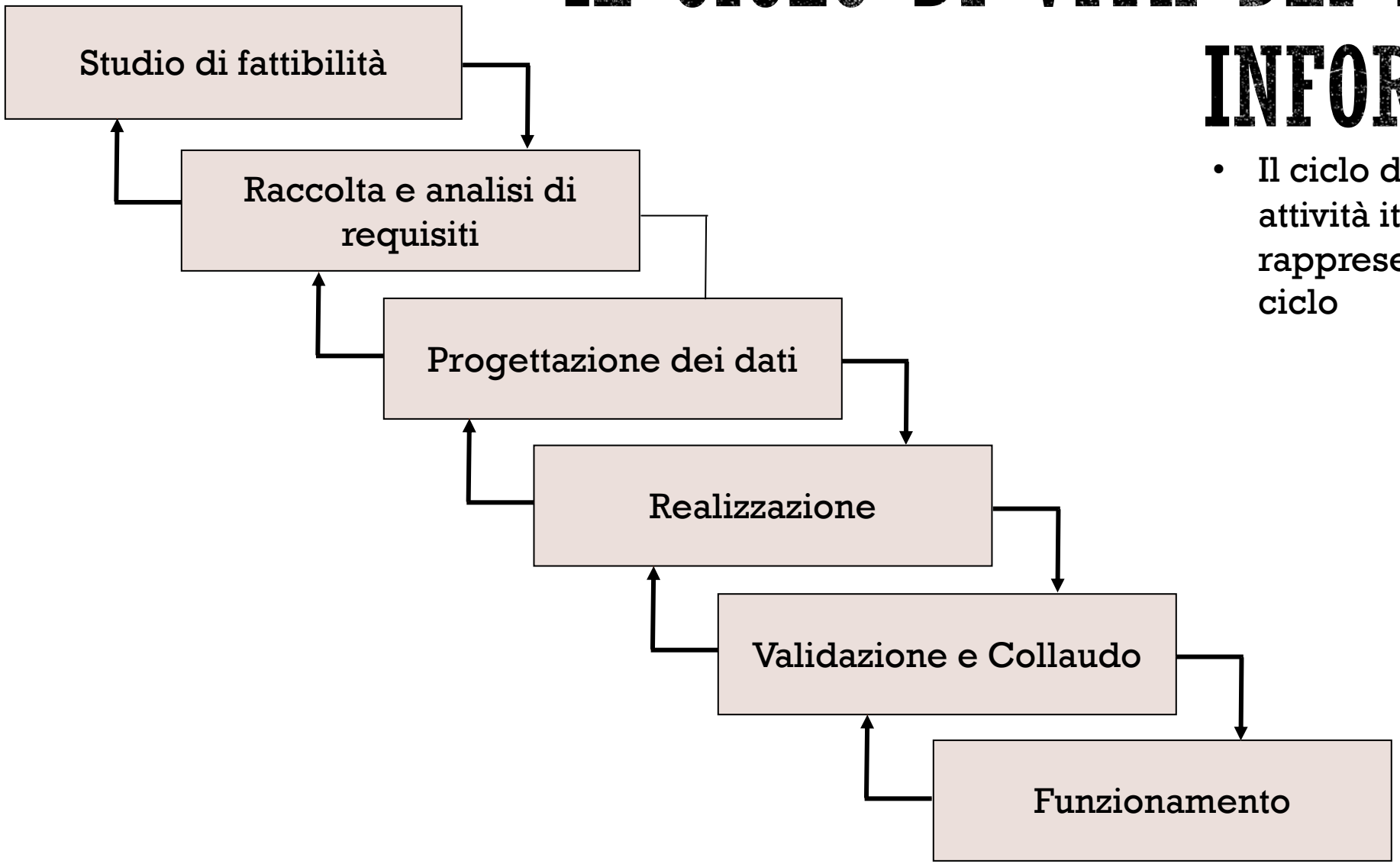
- È una delle attività del processo di sviluppo dei sistemi informativi.

- Va quindi inquadrata in un contesto più generale:
  - il ciclo di vita dei sistemi informativi.

- Riprendendo la definizione che abbiamo dato di sistema informativo nella prima lezione,
  - Componente (*sottosistema*)
  - di una organizzazione che gestisce (*acquisisce, elabora, conserva, produce*)
  - le informazioni di interesse (*cioè utilizzate per il perseguimento degli scopi dell'organizzazione*).



IL CICLO DI VITA DEI SISTEMI INFORMATIVI



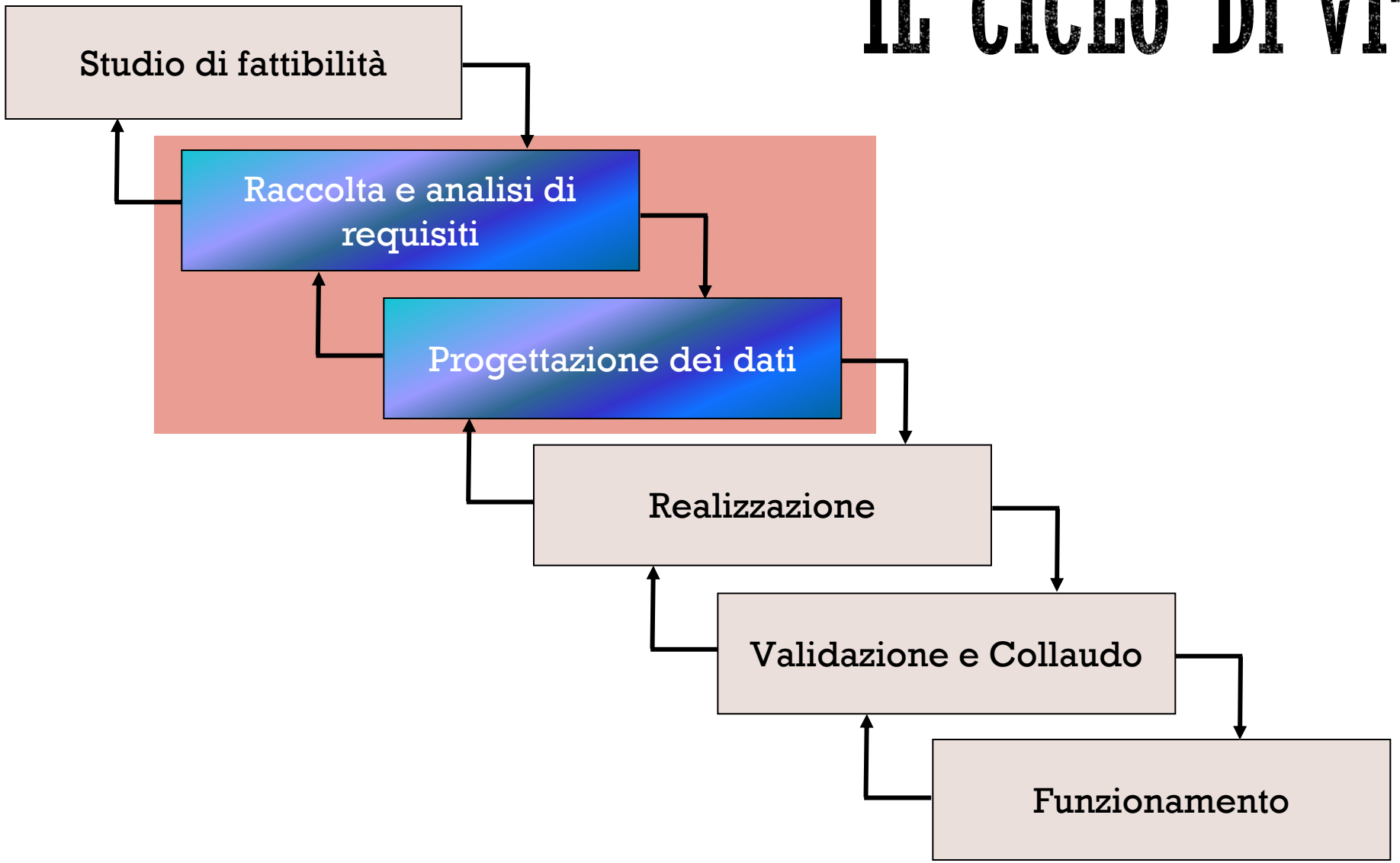
- Il ciclo di vita è una attività iterativa, rappresentata tramite un ciclo

FASI (TECNICHE) DEL CICLO DI VITA

- **Studio di fattibilità**
  - Si analizzano le potenziali aree di applicazione, si effettuano degli studi di *costi/benefici*, si determina la complessità di dati e processi, e si impostano le priorità tra le applicazioni.
- **Raccolta e analisi dei requisiti**
  - Comprende una raccolta dettagliata dei requisiti con interviste ai potenziali utenti, per definire le funzionalità del sistema.
- **Progettazione** di dati e funzioni
- **Realizzazione**
  - Si implementa il sistema informativo, si carica il DB e si implementano e si testano le transazioni.
- **Validazione e collaudo**
  - Si verifica che il sistema soddisfi i requisiti e le performance richieste.
- **Funzionamento**
  - La fase operativa del nuovo sistema parte quando tutte le funzionalità sono state validate.

Il rilascio può essere preceduto da una fase di addestramento del personale al nuovo sistema.
Se emergono nuovi funzionalità da implementare, si ripetono i passi precedenti, per includerle nel sistema (*manutenzione*).

IL CICLO DI VITA



LA METODOLOGIA DI PROGETTO

- Per garantire prodotti di buona qualità è opportuno seguire una metodologia di progetto, con:
  - Articolazione delle attività in fasi indipendenti tra loro;
  - Strategie da seguire nei vari passi e criteri di scelta (alternative);
  - Modelli di rappresentazione per descrivere i dati in ingresso e uscita delle varie fasi;
  - Proprietà;
  - Generalità;
  - Qualità del prodotto in termini di correttezza, completezza ed efficienza rispetto alle risorse impiegate;
  - Facilità d'uso delle strategie e dei modelli.

LA MODELLAZIONE

- La modellazione è un'attività fondamentale, per molteplici fasi nello sviluppo di un software
  - Nella fase di raccolta di requisiti serve per comunicare in maniera schematica con gli stakeholder del progetto.
  - Nella fase di progettazione è necessaria per definire gli stati del software e le transizioni da uno stato all'altro.
  - Nella fase di sviluppo è utile per definire i flussi di controllo dell'applicazione.
- Ogni fase utilizza dei diagrammi specifici per l'attività che si porta a termine



LA MODELLAZIONE DEI DATI

- Anche la progettazione dei dati passa attraverso diverse fasi di modellazione.

- Abbiamo due tipi principali di modelli:

1. **Modelli concettuali**: permettono di rappresentare i dati in modo indipendente da ogni sistema:
   - cercano di descrivere i concetti del mondo reale,
   - sono utilizzati nelle fasi preliminari di progettazione.
2. **Modelli logici**: utilizzati nei DBMS esistenti per l'organizzazione dei dati:
   - utilizzati dai programmi,
   - indipendenti dalle strutture fisiche.

   Esempi: relazionale, reticolare, gerarchico, a oggetti.



MODELLI CONCETTUALI, *PERCHÉ?*

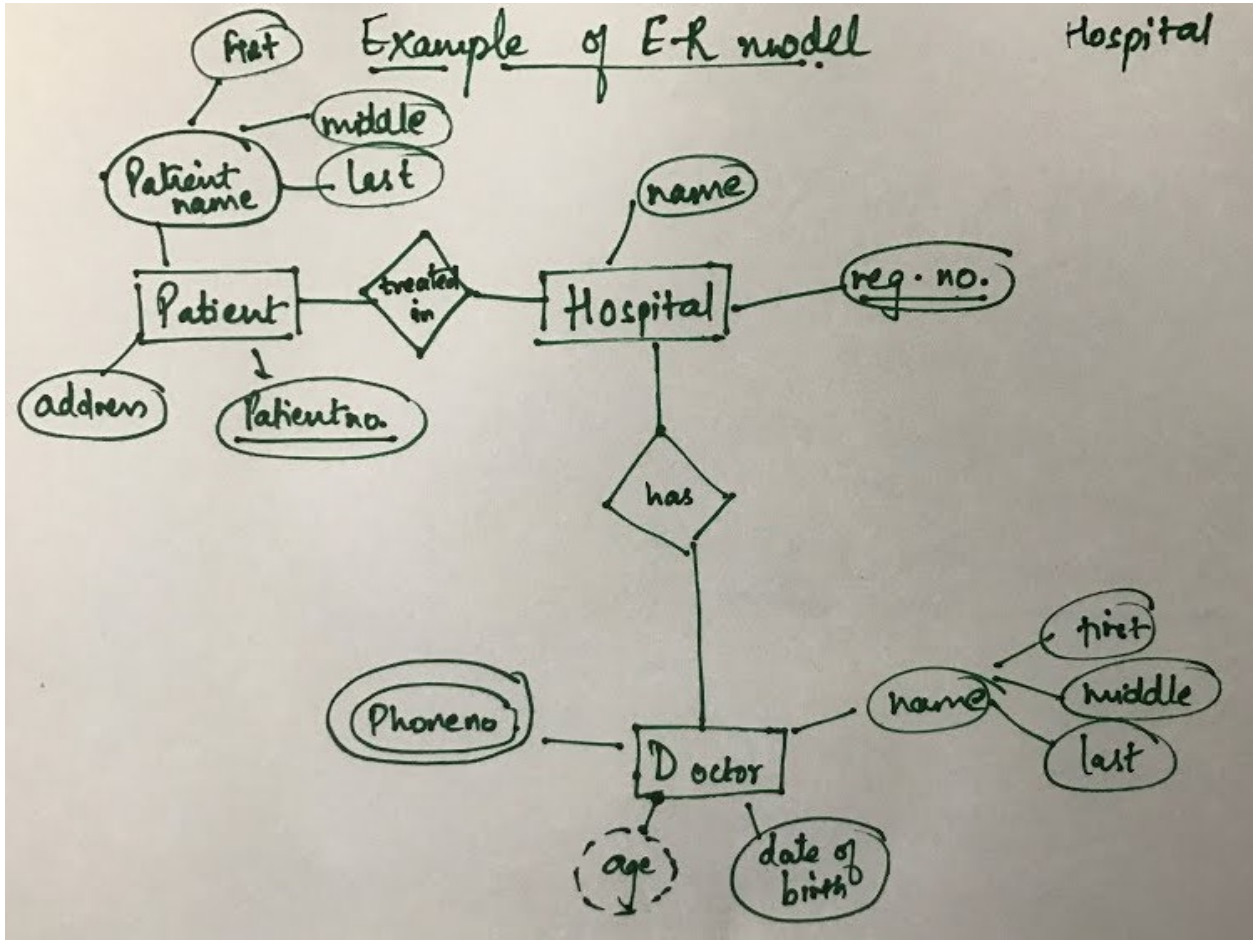
- La modellazione concettuale fornisce una buona base di partenza per la progettazione di buone basi di dati.

- Proviamo a modellare una applicazione definendo direttamente lo schema logico della base di dati:
  - da dove cominciamo?
  - rischiamo di perderci subito nei dettagli;
  - dobbiamo pensare subito a come correlare le varie tabelle (chiavi, relazioni, etc.);
  - i modelli logici sono rigidi.



MODELLI CONCETTUALI, *PERCHÉ?* (2)

- Servono per ragionare sulla realtà di interesse, indipendentemente dagli aspetti realizzativi.
- Permettono di rappresentare le classi di dati di interesse e le loro correlazioni.
- Prevedono efficaci rappresentazioni grafiche (utili anche per documentazione e comunicazione).



PROGETTAZIONE DI UNA BASE DI DATI



IL PROCESSO DI PROGETTAZIONE DEL DATABASE

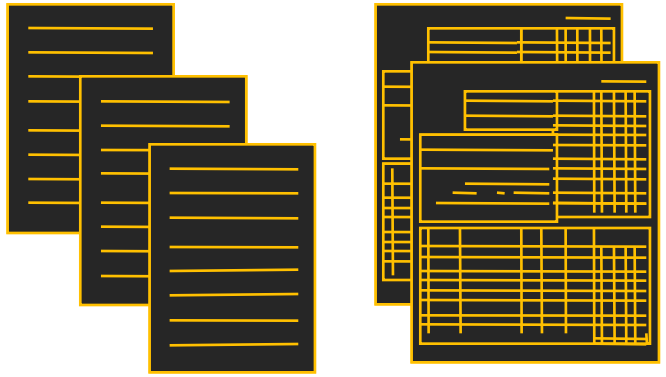
Il processo consta di quattro fasi:

1. Raccolta e analisi dei requisiti
2. Disegno del database concettuale (Progettazione Concettuale)
3. Disegno del database logico (Progettazione Logica)
4. Disegno del database fisico (Progettazione Fisica)



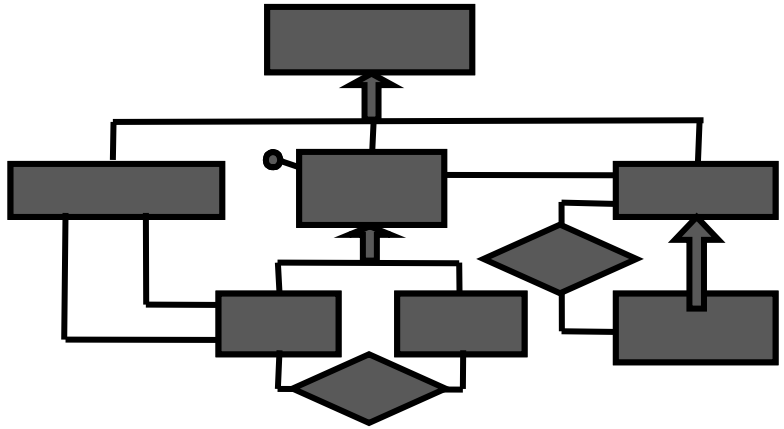

Requisiti della base di dati
Progettazione concettuale
"CHE COSA": analisi
Schema concettuale
Progettazione logica
Schema logico
"COME": progettazione
Progettazione fisica
Schema fisico

Requisiti



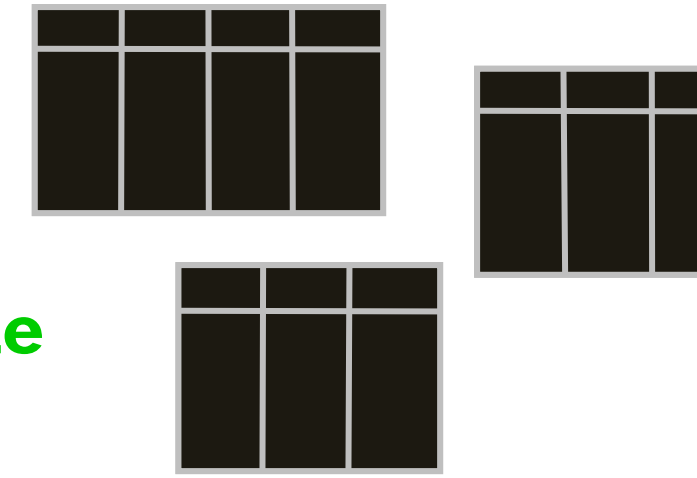
Progettazione concettuale

Schema Concettuale



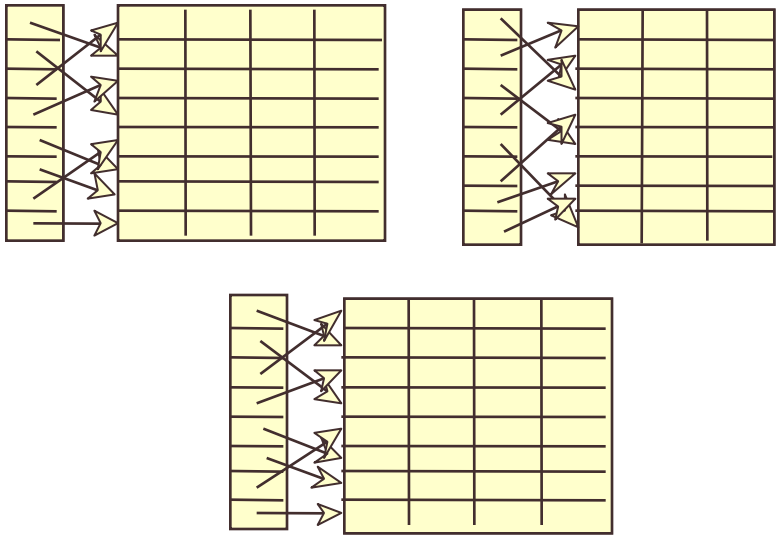
Progettazione logica

Schema Logico



Progettazione fisica

Schema Fisico



PROGETTAZIONE CONCETTUALE

- Individuare l'informazione rilevante da memorizzare per soddisfare le richieste informative e funzionali del caso applicativo
- Interesse specifico per la parte statica del problema: i dati
- Individuazione nel problema degli aspetti informativi rilevanti:
  - Gli elementi informativi di base (Entità)
  - Il modo in cui sono interrelati gli elementi informativi di base (Relazioni)
  - I vincoli che i dati memorizzati nel database dovrebbero soddisfare per garantire la qualità e la coerenza dell'informazione (Vincoli)
  - Le più importanti forme di interazione che gli utenti possono richiedere al database (ad esempio le interrogazioni della base di dati)
  - La frequenza con cui le interazioni avvengono
  - Il numero di utenti che simultaneamente possono interagire.



CARATTERISTICHE DELLA PROGETTAZIONE CONCETTUALE

- Dovrebbe essere **indipendente** da uno specifico modello dei dati e da uno specifico DBMS
  - Scelta da adottare successivamente

- **Deve escludere** dettagli implementativi legati a alla scelta del modello dei dati o alla scelta di uno specifico DBMS
  - Non siamo ancora nella fase in cui questa attività è eseguita



DOCUMENTAZIONE ALLA PROGETTAZIONE CONCETTUALE

- Per la progettazione concettuale si utilizzeranno i diagrammi Entity-Relationship (ER)
  - …ma vedremo anche i Class Diagram UML
- I diagrammi ER sono lo standard per la descrizione concettuale delle basi di dati
  - Sono comunque facilmente esprimibili mediante Class Diagram UML
- I Class Diagram di UML sono lo standard per la rappresentazione delle strutture dati nella programmazione ad oggetti
  - Sono indipendenti dal modello dei dati relazionale e agevolano le descrizione per il modello relazionale
- Dizionari delle entità e delle relazioni
  - Descrivono e commentano integrando la presentazione del diagramma ER
- Dizionario dei vincoli
- Facoltativo
  - Dizionario delle interrogazioni e indicazione della loro frequenza



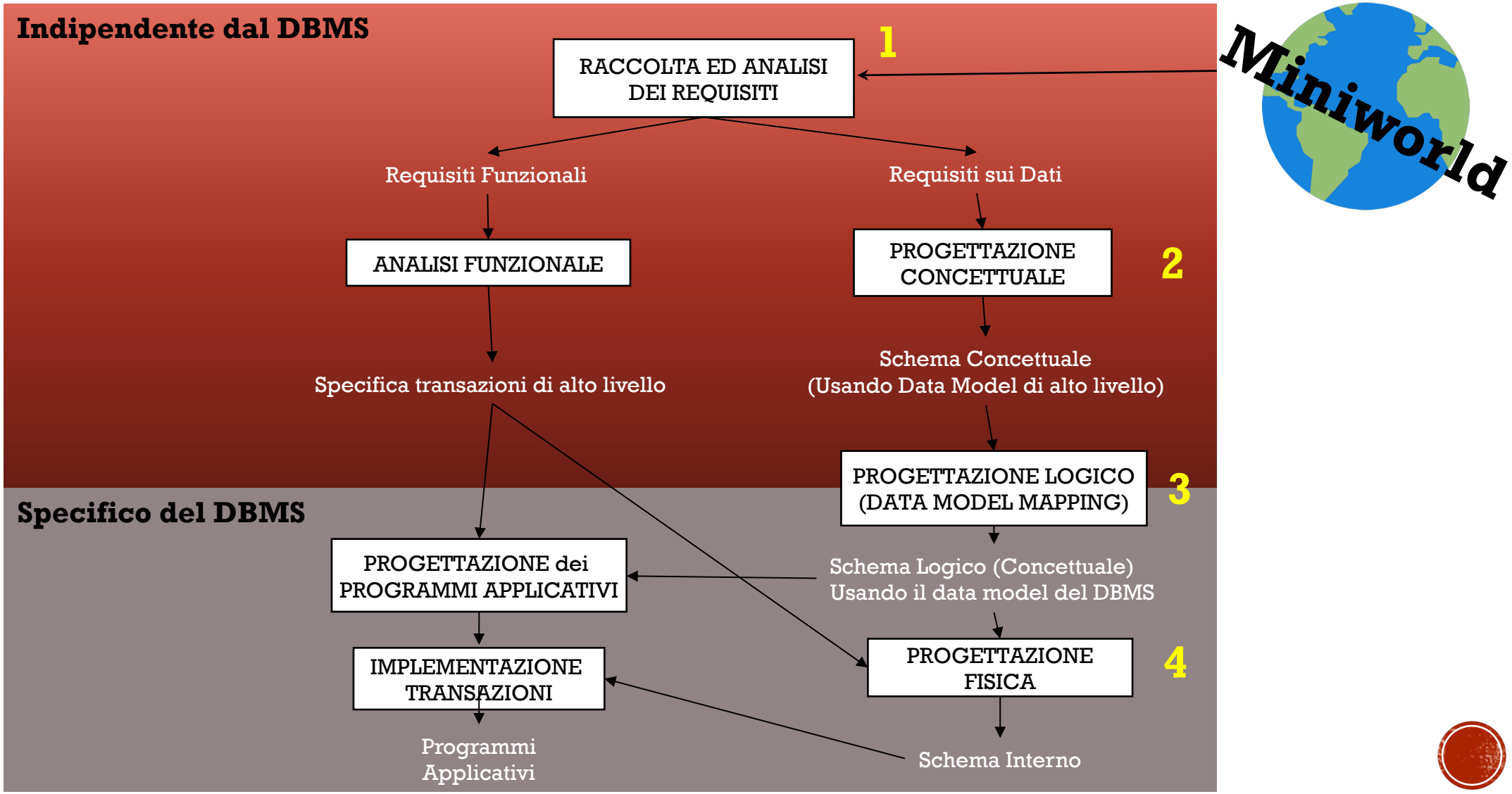
PASSI SUCCESSIVI ALLA PROGETTAZIONE CONCETTUALE

- Progettazione logica
  - Scelta del modello dei dati
  - Codifica delle entità ed associazioni nel modello dei dati

- Progettazione fisica
  - Scelta del DBMS
  - Definizione dei metadati utilizzando il DDL del DBMS
  - Implementazione dei vincoli di consistenza
  - Definizione dei dettagli implementativi utilizzando le funzionalità specifiche del DBMS scelto



LE FASI DI PROGETTAZIONE DI UN DATABASE



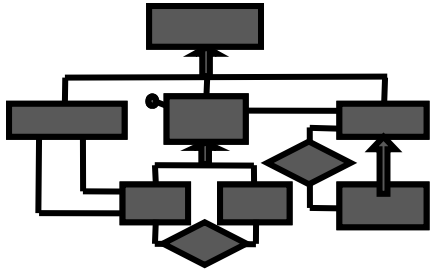
1. RACCOLTA E ANALISI DEI REQUISITI



- Il progettista del DB intervista i potenziali utenti dei DB per capire e documentare i requisiti utente.

- L'output di questa fase è duplice
  - Base di dati
    - Requisiti sui dati
  - Applicativo
    - Requisiti funzionali
      - Operazioni definite dall'utente (transazioni) che saranno applicate al DB (aggiornamenti, ricerche, …).
      - Specifica dei requisiti funzionali attraverso Diagrammi di flusso, Scenari, Diagrammi di sequenza, ….

2. DISEGNO DEL DATABASE CONCETTUALE



- Creazione dello schema concettuale, usando un data model concettuale ad alto livello.

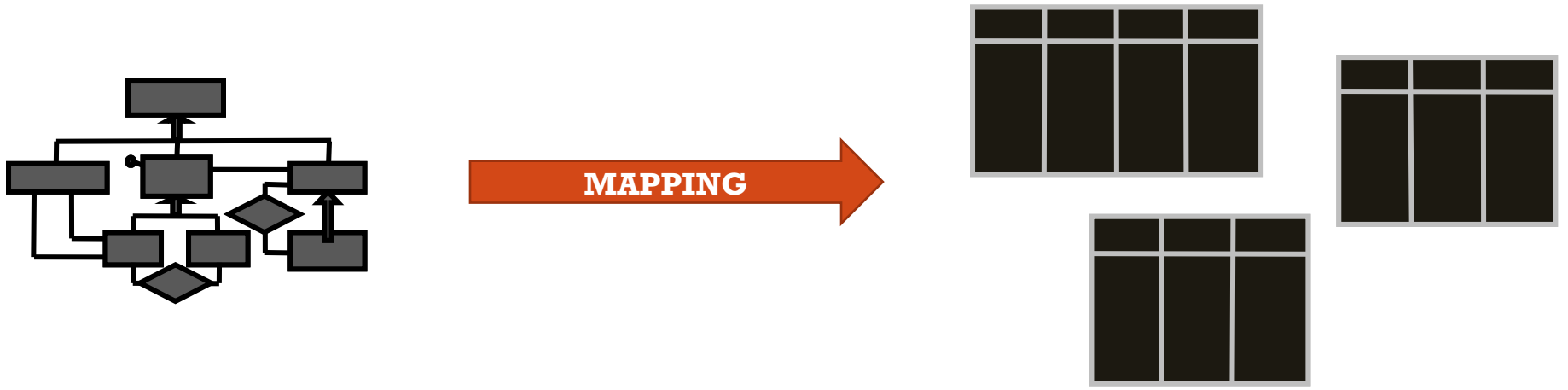
- *Output:*
  - Descrizione concisa dei requisiti utente dei dati inclusiva di una descrizione dettagliata di tipi di dati, relazioni e vincoli fatta usando i concetti del data model ad alto livello.
    - Poiché non vi sono dettagli implementativi, tale descrizione può essere usata per comunicare con utenti non tecnici.
    - Tale documentazione può essere usata anche come riferimento per assicurarsi di aver considerato tutti i requisiti dell'utente.
- Questo approccio abilita il progettista a concentrarsi sui dati ignorando i dettagli di memorizzazione.
- Dopo aver disegnato lo schema concettuale, le operazioni di base del data model possono essere usate per specificare le operazioni di alto livello individuate durante l'analisi funzionale (*passo 1*).

3. DISEGNO DEL DATABASE LOGICO

- Implementazione del database usando un DBMS commerciale.
  - Detto anche **data model mapping** (o **progettazione logica**).
  - Molti DBMS usano un data model di implementazione, in modo da trasformare facilmente lo schema concettuale da data model ad alto livello in data model di implementazione.

Output:

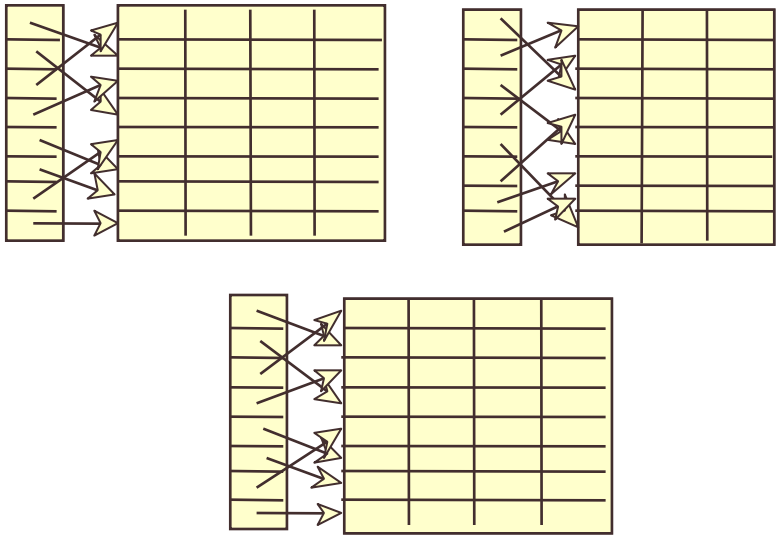
- Schema del database nel data model di implementazione specifico del DBMS scelto.



4. DISEGNO DEL DATABASE FISICO

- Specifica delle strutture di memorizzazione interne, degli access path e dell'organizzazione dei file.

- Progettazione dei programmi applicativi e implementazione delle transazioni.



IL PROCESSO DI PROGETTAZIONE DEL DATABASE

1. Raccolta e analisi dei requisiti
2. Disegno del database concettuale
3. Disegno del database logico
4. Disegno del database fisico

- La prima fase mal si presta all'utilizzo di rigorosi formalismi, essendo essenzialmente un grosso documento di testo.
- La seconda fase, invece, permette la definizione di modelli formali per supportare il lavoro del progettista.



MODELLAZIONE CONCETTUALE DEL DB «ER VS UML»



MODELLAZIONE DEL DATABASE

- L*a seconda fase, invece, permette la definizione di modelli formali per supportare il lavoro del progettista.*
- Abbiamo due tipologie di modelli che possiamo utilizzare
  - ER (Entity Relationship):
    - Il diagramma ER è definito proprio per la modellazione di basi di dati relazionali
    - I costrutti e i le forme utilizzate sono adattate alla modellazione di un database
  - UML (Unified Modelling Language)
    - I diagrammi UML sono utilizzati per molte attività di modellazione nel ciclo di vita di un progetto software
      - Sequence Diagram, Activity Diagram, StateChart Diagram, Class Diagram…
      - Ma è anche esso un linguaggio di modellazione (**UNIFICATO**) e può essere facilmente essere adattato anche per la descrizione di una base di dati
  - Nel corso vedremo entrambi i modelli.



MODELLO ER

- Il modello Entità-Relazione (ER) è un diffusissimo data model di alto livello, estesamente utilizzato per definire lo schema concettuale di un database.
- È stato concepito per essere più vicino ai concetti *"umani"*, e quindi facilmente comprensibile anche ad utenti non tecnici.
- Il modello ER ha avuto una grandissima diffusione principalmente per i formalismi grafici semplici e chiari che incorpora.
- Il modello ER descrive i dati con tre concetti fondamentali:
  - Entità
  - Attributi
  - Relazioni



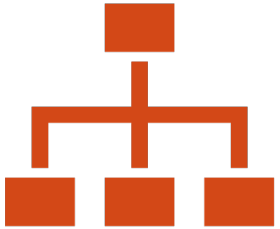
MODELLAZIONE DEI DATI IN UML

- UML (**U**nified **M**odeling **L**anguage) è un linguaggio grafico per la modellazione di applicazioni software basate sulla programmazione orientata agli oggetti.
  - Permette di rappresentare (attraverso una serie di diagrammi) tutti gli aspetti di un applicazione software: dati, operazioni, processi e architetture.

- UML può essere utilizzato in alternativa al modello ER:
  - I *diagrammi delle classi (Class Diagram)* sono usati per descrivere le classi di oggetti di interesse e le relazioni che intercorrono tra di esse.

- *Cambia la rappresentazione diagrammatica ma non l'approccio alla progettazione.*





UML

modellazione di un'applicazione software

aspetti strutturali e comportamentali (dati, operazioni, processi e architetture)

formalismo ricco

- diagramma delle classi, degli attori, di sequenza, di comunicazione, degli stati, …



ER/EER

modellazione di una base di dati

aspetti strutturali di una base di dati

costrutti funzionali alla modellazione di basi di dati

Principali differenze di UML rispetto ad ER

- assenza di notazione standard per definire gli identificatori
- possibilità di aggiungere note per commentare i diagrammi
- possibilità di indicare il verso di navigazione di una associazione
  - (non rilevante nella progettazione di una base di dati)
- Formalismi diversi

Il diagramma delle classi di un'applicazione è diverso dallo schema ER della base di dati della stessa applicazione

UML: LE ORIGINI

- Proposto a metà degli anni ’90 quale **formalismo unificante** per la modellazione o.o. di applicazioni software, è stato standardizzato sotto l’egida dell’Object Management Group (OGM).

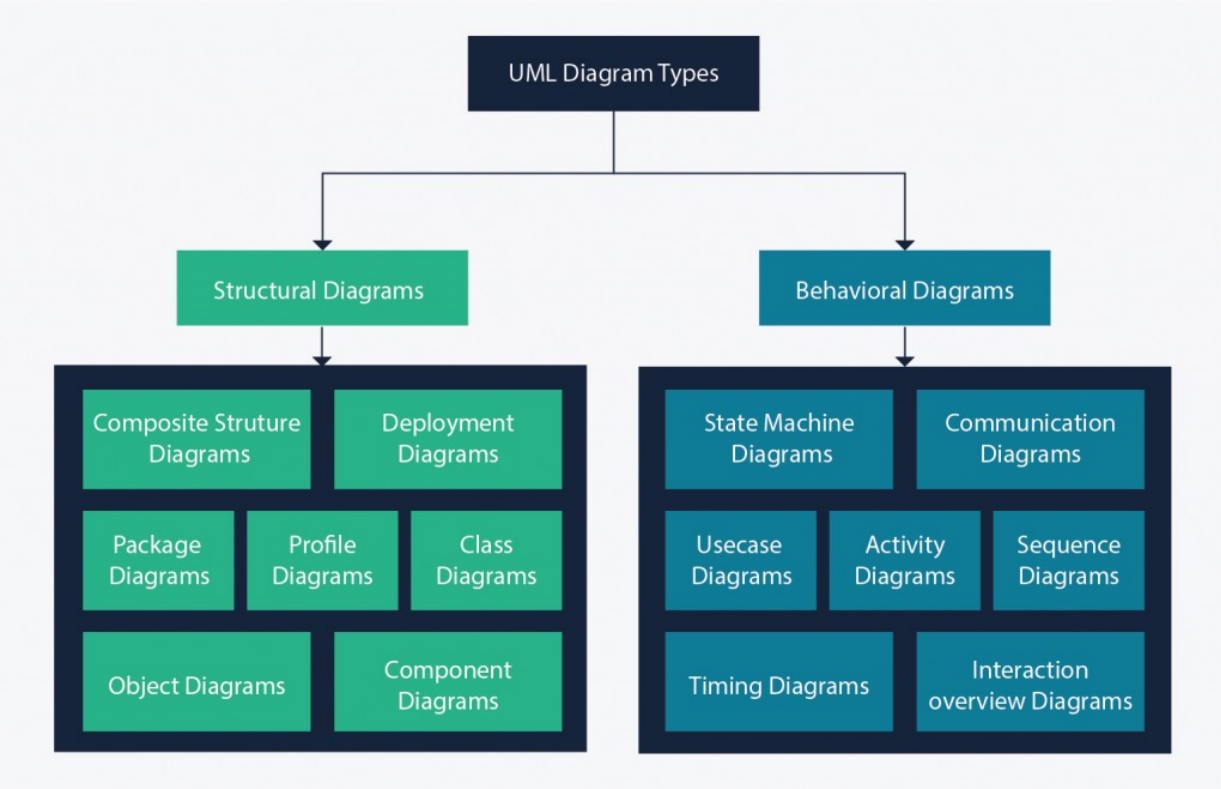
- UML offre una molteplicità di diagrammi, corredati da una descrizione testuale della loro semantica: **molteplicità di viste** della medesima applicazione.



UML: IL MODELLO DELL'APPLICAZIONE

- L'insieme dei diagrammi definisce il **modello dell'applicazione** (controparte dello schema ER):
  - UML è un metamodello per la descrizione di modelli di applicazioni software.

- Nella sua versione corrente UML prevede un certo numero (14) di **diagrammi fondamentali** che descrivono aspetti strutturali e comportamentali di un'applicazione



UML: I DIAGRAMMI PRINCIPALI

- **Diagramma delle classi**: descrive le caratteristiche statiche e dinamiche delle componenti (**classi**) e delle loro relazioni (**associazioni**).
- **Diagramma degli oggetti**: rappresentazione delle possibili istanze delle classi (**oggetti**) e dei loro collegamenti.
- **Diagramma dei casi d'uso**: modalità di utilizzo del sistema da parte di persone/altri sistemi (**attori**) e interazioni tra sistema e attori.
- **Diagramma di sequenza**: ordinamento temporale di messaggi (**invocazione di metodi**) scambiati tra i diversi oggetti dell'applicazione.
- **Diagramma delle attività**: comportamento dinamico di un processo dell'applicazione in termini dei flussi di **attività** da svolgere.
- **Diagramma degli stati**: descrive il ciclo di vita di un oggetto dell'applicazione attraverso gli **stati** che può assumere.
- **Diagramma di collaborazione (comunicazione)**: descrive lo scambio di **messaggi** tra gli oggetti, ma utilizzando notazione e prospettiva diverse.
- **Diagramma dei componenti**: descrive l'organizzazione delle **componenti** fisiche del sistema (file, moduli, …) e le loro dipendenze.
- **Diagramma di distribuzione dei componenti (deployment)**: descrivere la **dislocazione** dei nodi hardware del sistema e le loro associazioni.



MODELLAZIONE STRUTTURALE

- Modellazione strutturale:
  - Rappresenta una vista di un sistema software che pone l'accento sulla struttura degli oggetti (classi di appartenenza, relazioni, attributi, operazioni)

- Il **diagramma delle classi** descrive il tipo degli oggetti che compongono il sistema e le relazioni statiche esistenti tra loro



ESEMPIO: IL DB COMPANY



REQUISITI PER IL DATABASE COMPANY

- La compagnia è organizzata in DIPARTIMENTI. Ogni Dipartimento ha un nome, un numero ed un impiegato che lo gestisce. Bisogna tener traccia della data di insediamento del manager. Un dipartimento può avere più locazioni.

- Ogni dipartimento controlla una serie di PROGETTI. Ogni progetto ha un nome, un numero ed una singola locazione.

- Per IMPIEGATO si tiene traccia di: nome, SSN, indirizzo, salario, sesso e data di nascita. Ogni impiegato lavora per un dipartimento e può lavorare su più progetti. Teniamo traccia del numero di ore settimanali che un impiegato spende su un progetto e del supervisore di ogni impiegato.

- Ogni impiegato ha una serie di PERSONE A CARICO. Per ogni persona a carico, registriamo: nome, sesso, data di nascita e parentela con l'impiegato.



ENTITÀ, ATTRIBUTI E CHIAVI

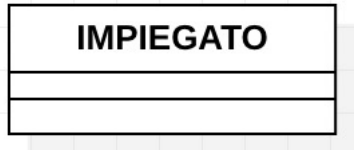


ENTITÀ

- Le **entità** corrispondono a classi di oggetti del mondo reale (fatti, persone, …) che hanno proprietà omogenee, comuni ed esistenza *"autonoma"* ai fini dell'applicazione di interesse.
- Un'entità E può essere un oggetto fisico (casa, impiegato, …) o un oggetto concettuale (un lavoro, una società, …).



Rappresentazione ER dell'entità *"Impiegato"*

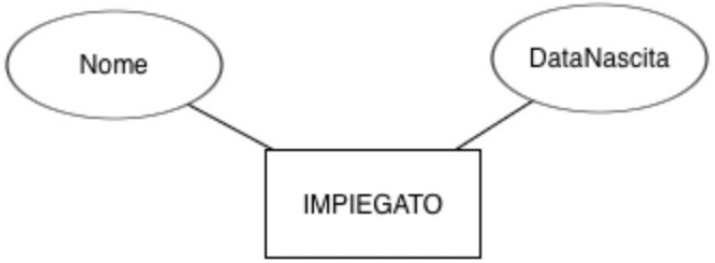


In UML, l'entità è chiamata *Classe*

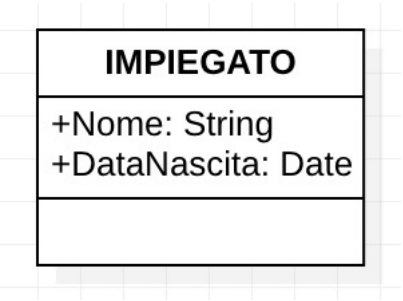
Rappresentazione UML della classe *"Impiegato"*

ATTRIBUTI

- Ogni entità E ha delle proprietà dette **attributi A_1 , A_2 , A_3 , … ,A_n**.
  - **E (A_1, A_2, A_3, …, A_n)**
  - *Es:* l'entità *Impiegato* ha attributi **nome, cognome**, **età**, **indirizzo**, **salario**, **telefono**, …
- Ogni entità è caratterizzata da un set di valori per i suoi attributi.
  - Nel diagramma ER, un attributo descrive una proprietà elementare di un'entità o di una relazione.
  - Nel Class Diagram, un attributo rappresenta una proprietà elementare degli oggetti di una classe.



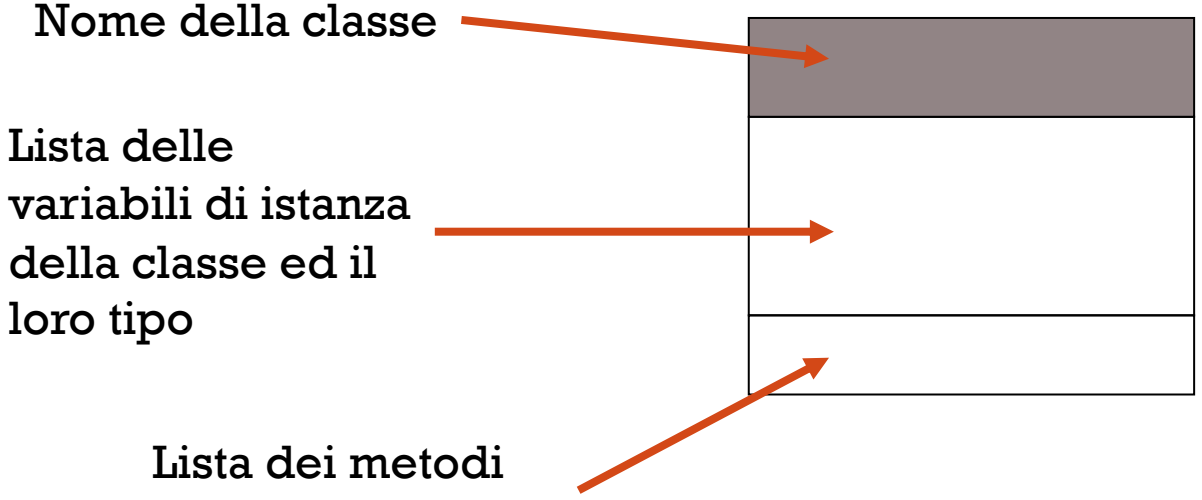
Rappresentazione ER dell'entità *"Impiegato"* con gli attributi *"Nome"* e *"DataNascita"*

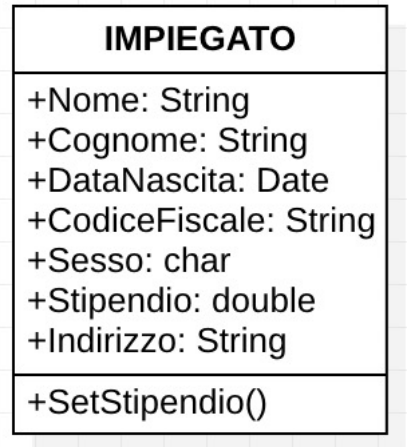


Rappresentazione UML della classe *"Impiegato"* con gli attributi *"Nome"* e *"DataNascita"*

RAPPRESENTAZIONE DI DATI CON I DIAGRAMMI DELLE CLASSI

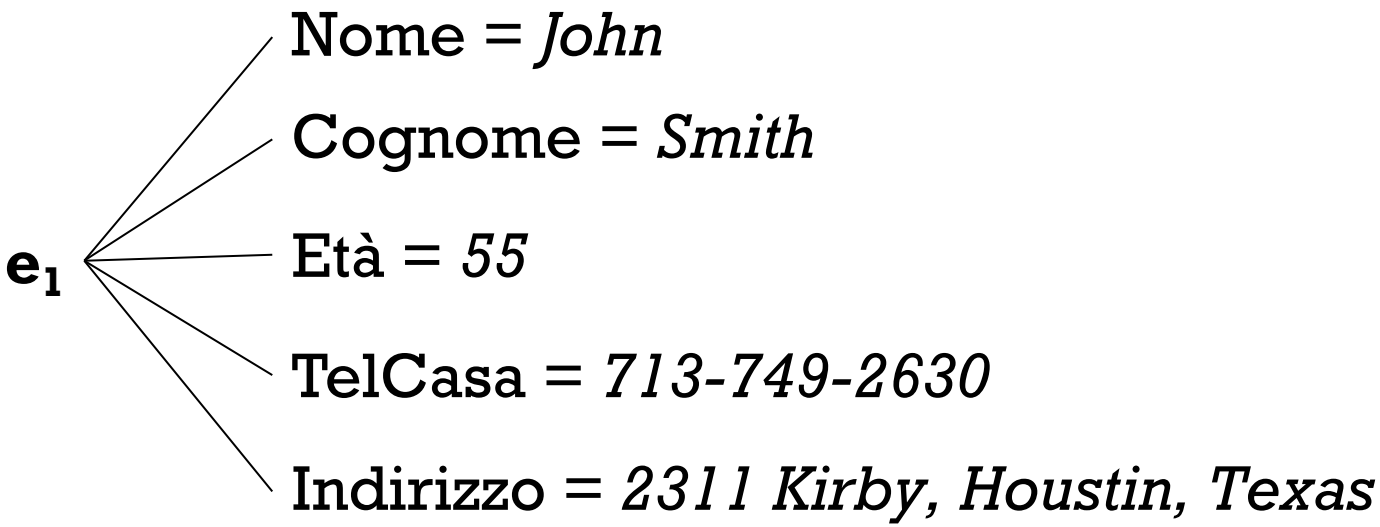
- **Classi:** Sono le componenti principali dei diagrammi delle classi.
  - Corrispondono alle entità del modello ER.
  - È possibile aggiungere i metodi (i.e., le operazioni ammissibili sugli oggetti della classe).
  - I dati vengono descritti *insieme* alle operazioni da svolgere su di essi.





ENTITÀ E ATTRIBUTI

- ***Esempio:*** entità *Impiegato*



TIPI DI ATTRIBUTI

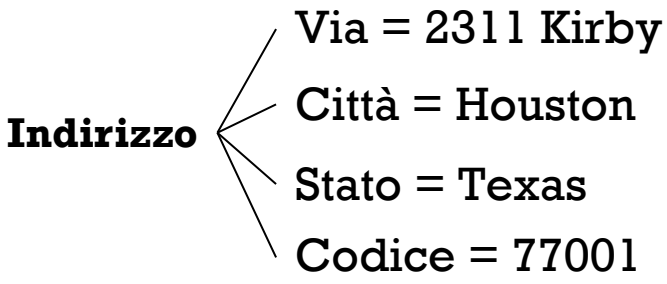
Gli attributi possono essere di più tipi

Divisibile?	Più valori?	Calcolabile?
Semplice	Single-valued	Archiviato
Composto	Multivalued	Derivato



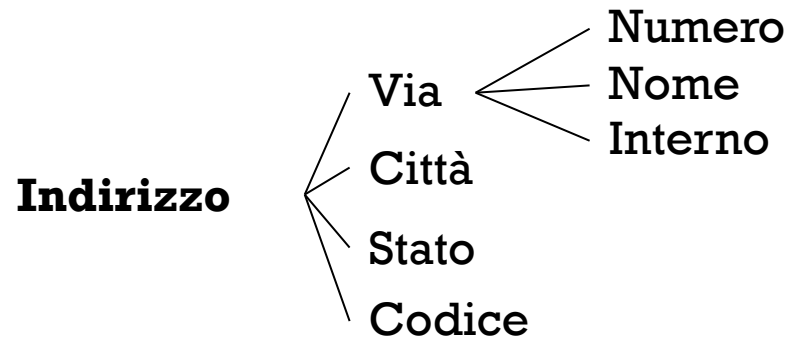
ATTRIBUTI SEMPLICI E COMPOSTI

- **Attributi semplici**: ogni entità ha un valore singolo (atomico) per tale attributo.
- **Attributi composti**: possono essere divisi in sottoparti, che rappresentano informazioni di base con loro significati indipendenti.
  - ***Es.*** l'attributo **Indirizzo** dell'entità **IMPIEGATO:**
  - Denotato come *Indirizzo(Via, Città, Stato, Codice)*



ATTRIBUTI SEMPLICI E COMPOSTI (2)

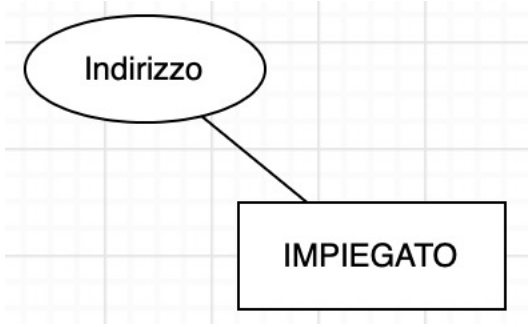
- Gli attributi composti possono formare una gerarchia:

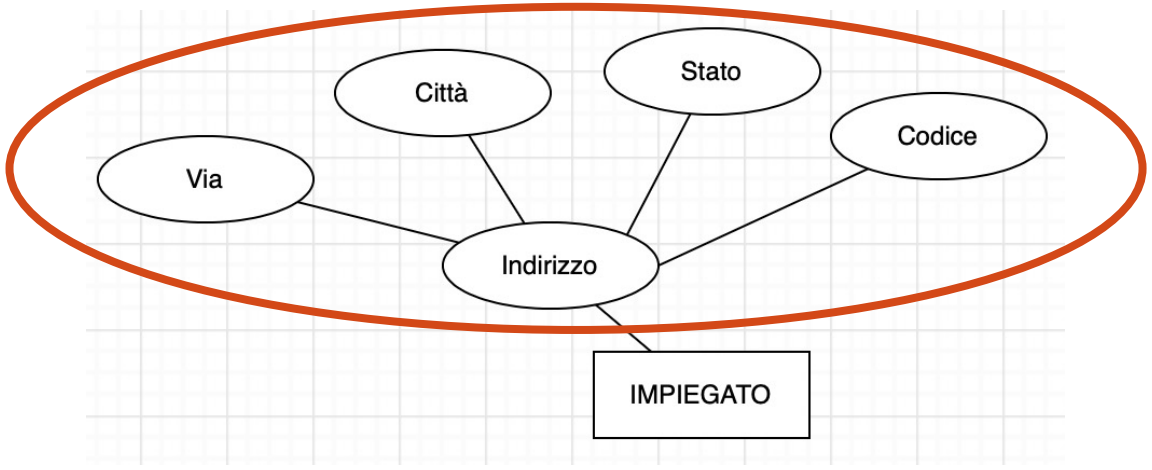


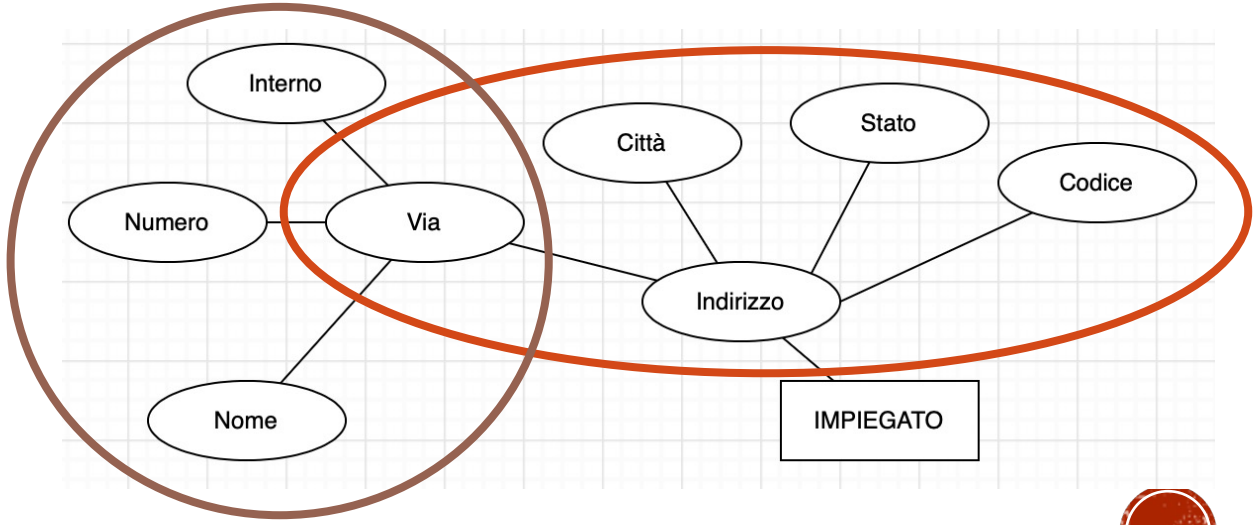
Indirizzo(Via(Numero, Nome, Interno), Città, Stato, Codice)

- L'utilizzo di un attributo semplice o di uno composto dipende dalla necessità o meno di trattare separatamente le sottoparti.

ATTRIBUTI SEMPLICI E COMPOSTI (ER)

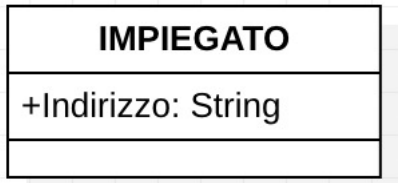


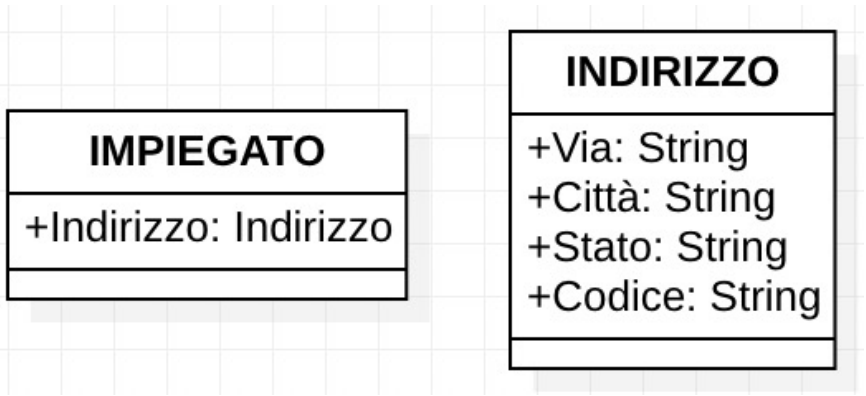


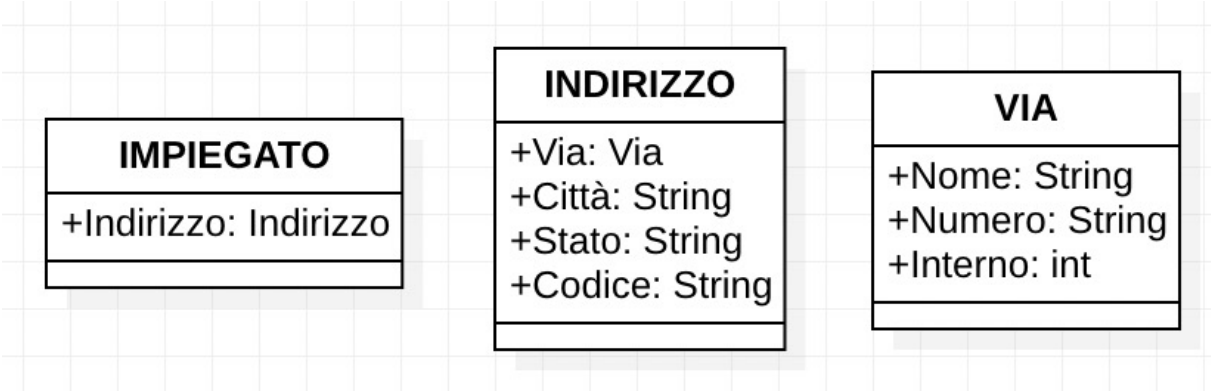


ATTRIBUTI SEMPLICI E COMPOSTI (UML)

- In UML non c'è una notazione specifica per la rappresentazione di attributi multivalore



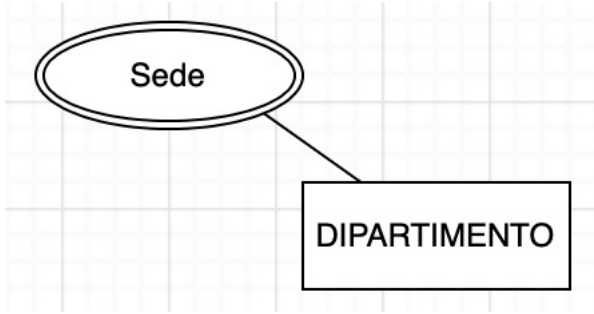




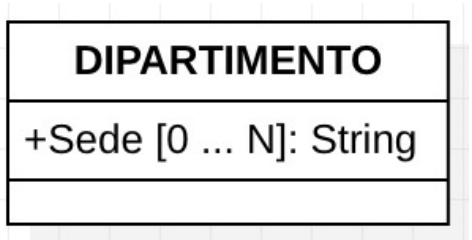
SINGLE-VALUED E MULTIVALUED

- *Attributi single-valued*:
  - hanno un solo valore per ciascuna entità.
  - ***Es:*** l'età di un impiegato.

- *Attributi multivalued*:
  - può avere un insieme di valori per la stessa entità.
  - ***Es:*** l'attributo **Sede** per un'entità Dipartimento.
    - Denotato come Dipartimento({Sede})
  - In UML, può essere determinato un limite inferiore e un limite superiore al numero di valori per un'entità.



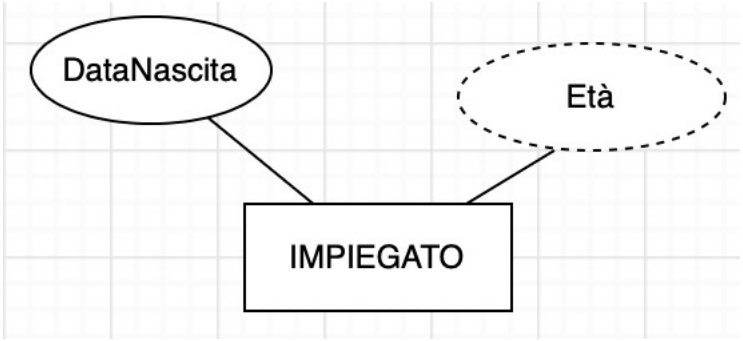
ER



UML

ARCHIVIATI E DERIVATI DA ALTRI ATTRIBUTI

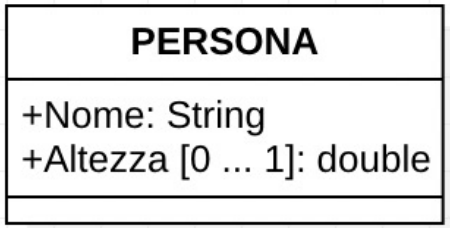
- *Attributi derivati:* alcuni attributi possono essere in relazione tra loro.
  - ***Es:*** l'età e la data di nascita di un impiegato:
    - DataNascita **attributo memorizzato**
    - Età **attributo derivato**
- Alcuni valori di attributi possono essere derivati da entità correlate.
  - ***Es:*** NumeroImpiegati di una entità *"dipartimento"* può essere derivato contando il numero di impiegati relati al (che lavorano per) dipartimento





VALORI NULLI

- Alcuni attributi possono avere valore **null** col significato di *"non noto"* o *"mancante" o "non applicabile"*.
  - L'attributo può esistere ma non essere noto: **Es.** l'altezza di una persona
  - L'attributo può non esistere: **Es.** l'interno di un appartamento per una casa unifamiliare
- Nel diagramma ER non c'è una vera e propria notazione per rappresentare i valori **NULL.**
- Nel diagramma UML si può utilizzare la stessa notazione per gli attributi multi-valore.





ATTRIBUTI COMPLESSI

- In alcuni casi, gli attributi composti e gli attributi multi-valore possono essere annidati in modo arbitrario
  - **Es.** Una persona può avere più di un indirizzo ed ogni indirizzo può avere più di un telefono, e sia telefono che indirizzo sono attributi composti

Persona (Nome, Cognome, DataN, { IndirizzoTelefono ({ Telefono (Prefisso, NumTelefono) } , Indirizzo (IndirizzoVia (Numero, Via, NumeroAppartamento), Città, Stato, CAP)) })

Per la modellazione si utilizzano le convenzioni mostrate prima per gli attributi composti e quelli multi-valore.



TIPI E INSIEMI DI ENTITÀ E CHIAVI



INTENSIONE ED ESTENSIONE

- Entità con gli stessi attributi di base sono raggruppati in un tipo di entità.
- ***Esempio:*** Tutte le persone che lavorano per un dipartimento possono essere definite con l'entità *Impiegato*.

Nome del Tipo di Entità:
(Schema o Intenzione)

IMPIEGATO

Un tipo di entità descrive lo schema (o intenzione) per un insieme di entità che possiedono gli stessi attributi

Insieme di Entità
(Estensione)

Nome, Cognome, Età, Stipendio

e_1 ■
(John, Smith, 55, 80k)

e_2 ■
(Fred, Brown, 40, 30k)

e_3 ■
(Judy, Clark, 25, 20k)

⋮

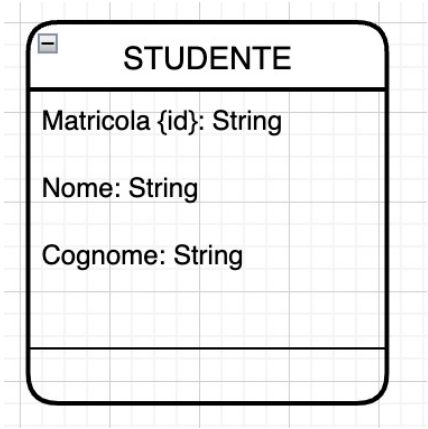
Le entità individuali di un particolare tipo di entità sono raggruppate in una collezione o insieme detta estensione del tipo di entità.

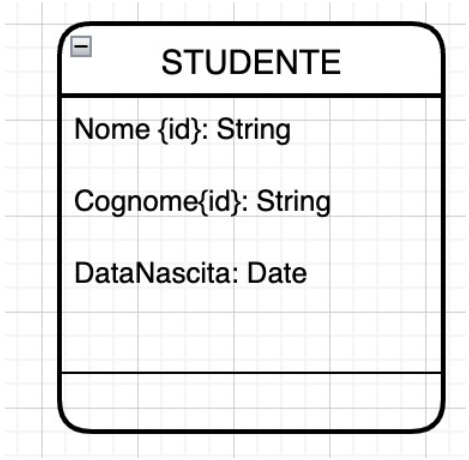


ATTRIBUTI CHIAVE DI UN TIPO DI ENTITÀ

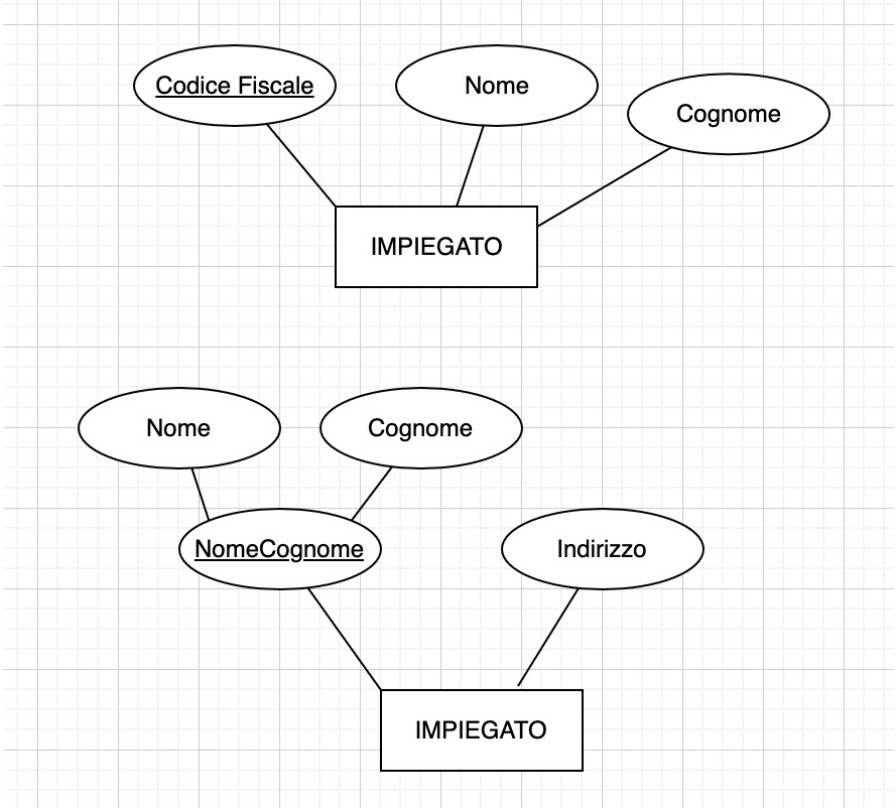
- Un importante vincolo sulle entità di un tipo di entità è la chiave o vincolo di unicità.
  - L'attributo chiave di un tipo di entità è un attributo che deve avere un valore univoco per cui ogni entità.
  - Serve per identificare univocamente una singola entità
  - ***Es.:*** Il codice fiscale di una persona.
- Talvolta più attributi insieme formano una chiave:
  - In tal caso tali attributi possono essere raggruppati in un ***attributo composto*** che diventa chiave.
- Alcuni tipi di entità possono avere più di un attributo chiave.
- Il vincolo di chiave per un'entità proibisce a qualsiasi coppia di entità di avere contemporaneamente lo stesso valore per l'attributo chiave.
- **Il vincolo di chiave deriva dai vincoli del mini-mondo**
- Vi possono essere entità che non hanno attributi chiave
  - In tal caso questo tipo di entità è chiamato *entità debole* (che vedremo nel seguito)

ATTRIBUTI CHIAVE IN UML E ER

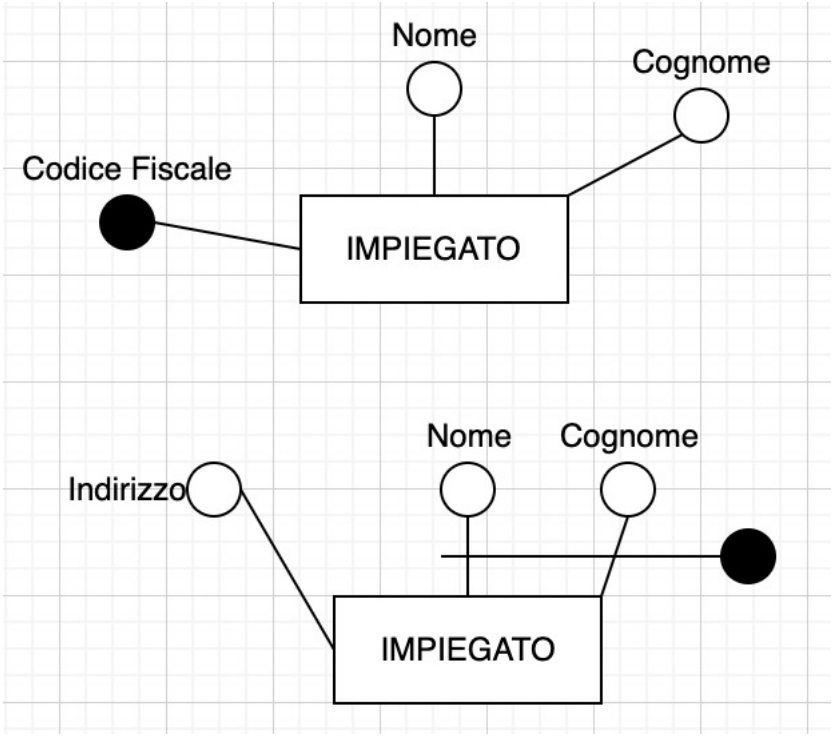




UML



ER
(Elmasri, Navathe)



ER
(Atzeni, Ceri, Paraboschi, …)

ESEMPIO: ENTITÀ AUTO

AUTO

Targa(Numero,Provincia), Telaio, Marca, Modello, Anno_imm, {Colore}

Car 1 ·

((ASC 123, TEXAS), tk629, Ford Mustang, convertible, 1989, {red, black})

Car 2 .

((ABO 123, NEW YORK), WPS872, N~san sontra 2~cor, 1992, {blue})

Car3 ·

((VYS 720, TEXAS), TD729, Chrysler LeBaron, 4dcor, 1993, {white, bue})

Gli attributi multivalued sono mostrati tra parentesi **{ }.**
Le componenti di un attributo composto sono mostrate tra parentesi **().**

DOMINIO DI UN ATTRIBUTO

- Ciascun attributo semplice è associato a un dominio o insieme di valori che rappresenta l'insieme dei valori che l'attributo può assumere.
- **Esempio:** l'età dell'impiegato può variare nel dominio (16, 70).
- Matematicamente:
  - Un attributo semplice A, del tipo di entità E, avente dominio V, è una funzione:
    A: E → P(V) P(V) insieme potenza dei sottoinsiemi di V:
    Un valore **null** è rappresentato con **Ø**.

- Per un attributo singolo
  - A(e) il valore dell'attributo A per l'entità e. A(e) viene detto **singleton**

- Per un attributo composto A, del tipo di entità E, il dominio V è il prodotto cartesiano

$$V = P(V_1) \times P(V_2) \times \ldots \times P(V_n)$$

dove V_i è il dominio dell'attributo semplice *i*-mo di A,

REQUISITI PER IL DATABASE COMPANY

- La compagnia è organizzata in DIPARTIMENTI. Ogni Dipartimento ha un nome, un numero ed un impiegato che lo gestisce. Bisogna tener traccia della data di insediamento del manager. Un dipartimento può avere più locazioni.

- Ogni dipartimento controlla una serie di PROGETTI. Ogni progetto ha un nome, un numero ed una singola locazione.

- Per IMPIEGATO si tiene traccia di: nome, SSN, indirizzo, salario, sesso e data di nascita. Ogni impiegato lavora per un dipartimento e può lavorare su più progetti. Teniamo traccia del numero di ore settimanali che un impiegato spende su un progetto e del supervisore di ogni impiegato.

- Ogni impiegato ha una serie di PERSONE A CARICO. Per ogni persona a carico, registriamo: nome, sesso, data di nascita e parentela con l'impiegato.



DISEGNO CONCETTUALE DEL DATABASE COMPANY

- Descriviamo i tipi di entità per il database COMPANY.
- In accordo ai requirement possiamo identificare quattro tipi di entità:

1. DIPARTIMENTO
   Nome, Numero, {Sedi}, Manager, Datains_manager

   Nome e Numero sono entrambi attributi chiave.

2. PROGETTO
   Nome, Numero, Luogo, Dip_controllo

   Nome e Numero sono entrambi attributi chiave.



DISEGNO CONCETTUALE DEL DATABASE COMPANY (2)

3. IMPIEGATO
   Nome, SSN, Sesso, Indirizzo, Stipendio, DataNascita, Dipartimento, Supervisore

   SSN è un attributo chiave.

   Nome e Indirizzo sono attributi composti (occorrerebbe verificare con l'utente se ha bisogno di riferire ai componenti individuali).

4. PERS_A_CARICO
   Sesso, DataNascita, Impiegato, Nome_pers_carico, Parentela



DISEGNO CONCETTUALE DEL DATABASE COMPANY (3)

- Dobbiamo però rappresentare:
  - Il fatto che un impiegato può lavorare su più progetti.
  - Il numero di ore settimanali di un impiegato su ciascun progetto.
- Si può aggiungere un attributo a IMPIEGATO *"**Lavora_su**"* composto di due componenti semplici (**Progetto**, **Ore**):

IMPIEGATO
Nome (FName, Minit, LName), SSN, Sesso, Indirizzo, Stipendio, DataNascita, Dipartimento, Supervisore, {Lavora_su(Progetto, Ore)}

- In alternativa, le stesse informazioni si potrebbero mantenere nel tipo di entità PROGETTO con un attributo composto:
  - **Addetti** (**Impiegato**, **Ore**)



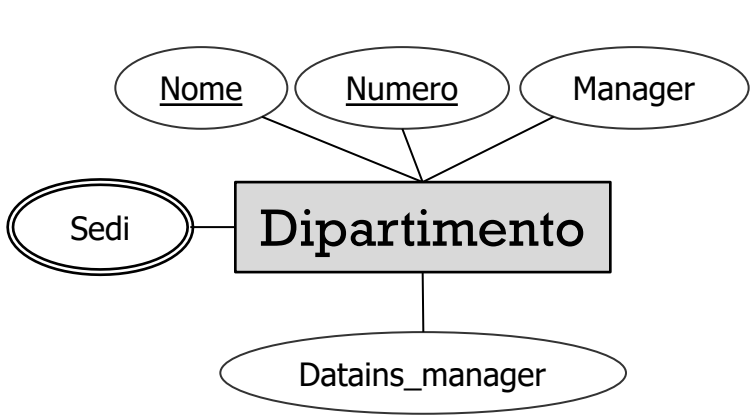
DISEGNO CONCETTUALE DEL DATABASE COMPANY (4)

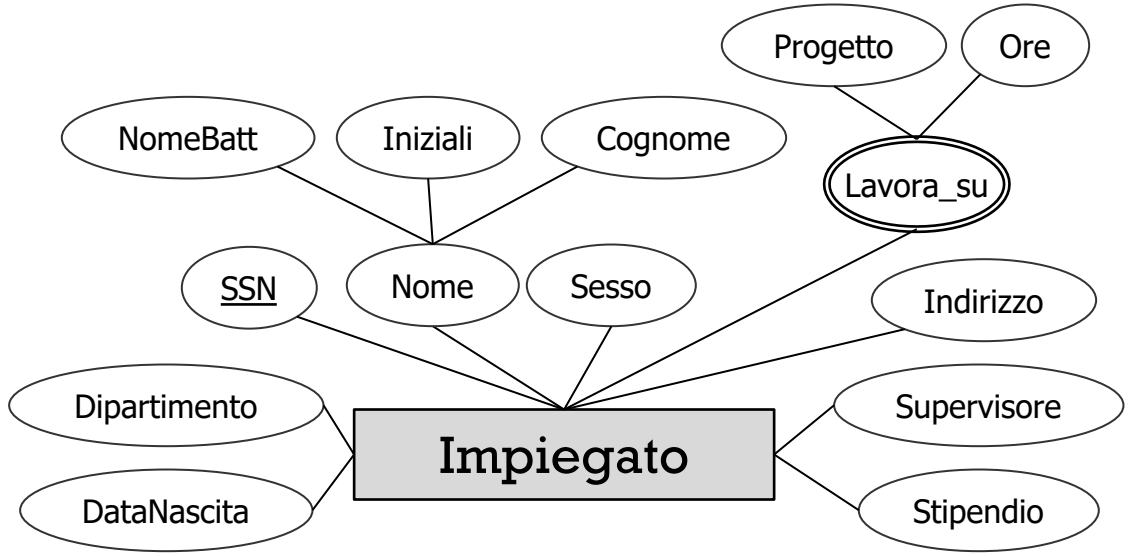
- Esistono varie relazioni implicite:
  - L'attributo Manager di DIPARTIMENTO si riferisce a un impiegato che gestisce il dipartimento;
  - L'attributo Dip_controllo di PROGETTO si riferisce al dipartimento che controlla il progetto;
  - L'attributo Dipartimento di IMPIEGATO si riferisce al dipartimento per cui lavora l'impiegato;
  - …

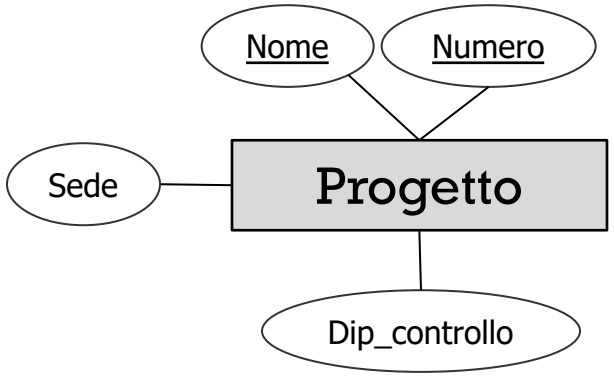
- Nel disegno iniziale queste associazioni tra entità sono rappresentabili come attributi, ma durante il processo di raffinamento nel modello ER questi riferimenti dovrebbero essere rappresentati come relazioni.

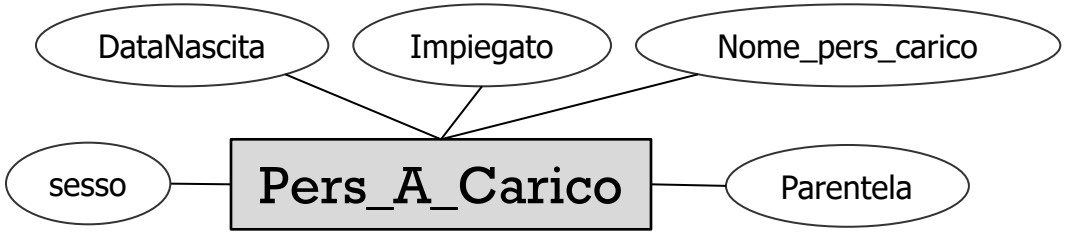


PROGETTAZIONE ER PRELIMINARE PER IL DB COMPANY

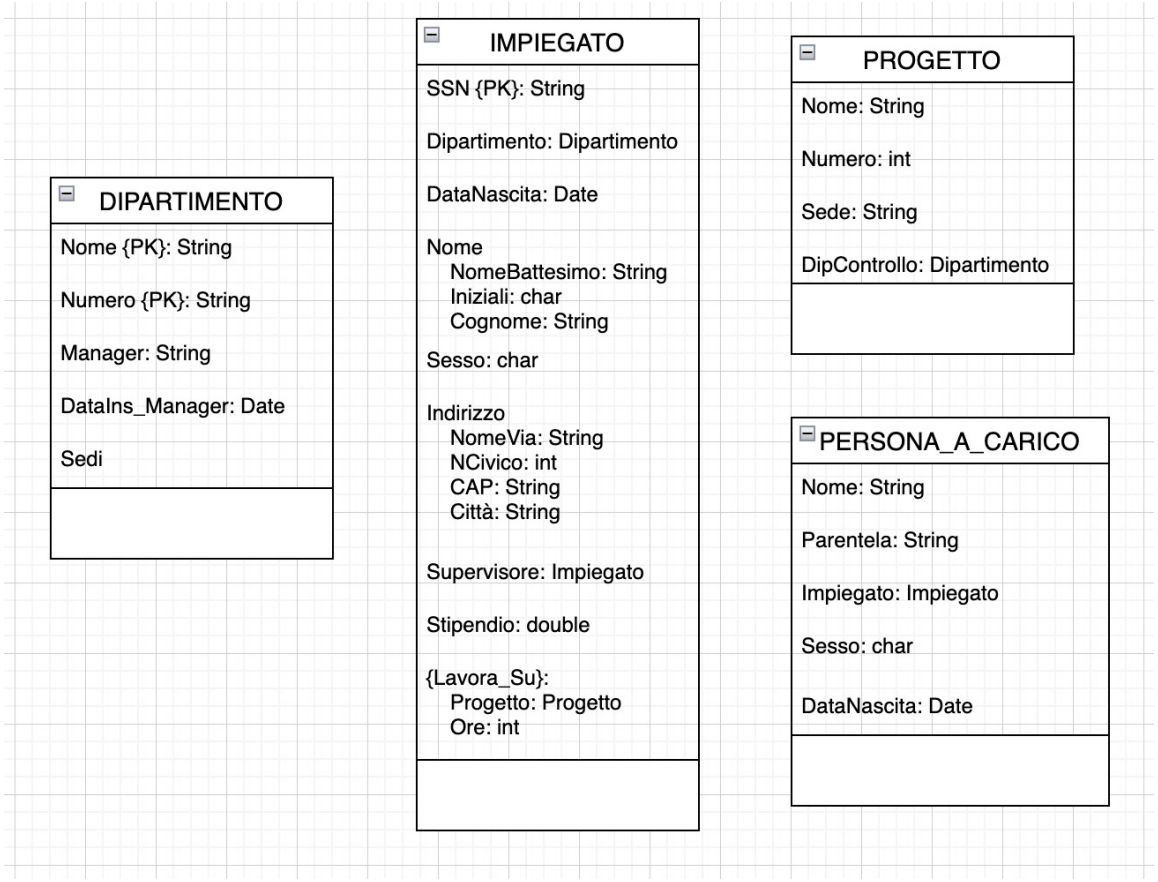








PROGETTAZIONE UML (CLASS DIAGRAM) PRELIMINARE PER IL DB COMPANY



RELAZIONI, RUOLI E VINCOLI (ER)



TIPI E ISTANZE DI RELAZIONI

- Le relazioni corrispondono a legami logici tra entità, significativi ai fini dell'applicazione di interesse.

- Un **tipo di relazione R** è un'associazione tra *n* tipi di entità $\mathbf{E_1}$, $\mathbf{E_2}$,…,$\mathbf{E_n}$.

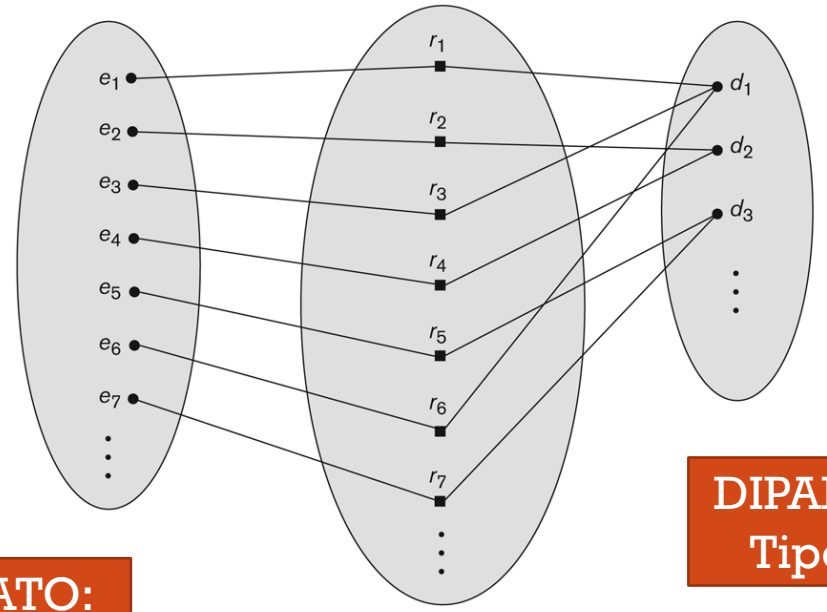
- Le occorrenze o istanze di relazione associano n entità dei tipi di relazione richiesti.

- Ogni tipo di entità è detto partecipare al tipo di relazione.



TIPI E ISTANZE DI RELAZIONI (2)

- ***Es.:*** vogliamo rappresentare il fatto che ogni impiegato e_i lavora per un dipartimento d_j.
- Definiamo il tipo di relazione LAVORA_PER tra i due tipi di entità IMPIEGATO e DIPARTIMENTO: ogni relazione r_i associa una entità IMPIEGATO e_i con una entità DIPARTIMENTO d_j.



IMPIEGATO: Tipo di Entità

LAVORA_PER: Relazione

DIPARTIMENTO: Tipo di Entità

TIPI E ISTANZE DI RELAZIONI (3)

Matematicamente:

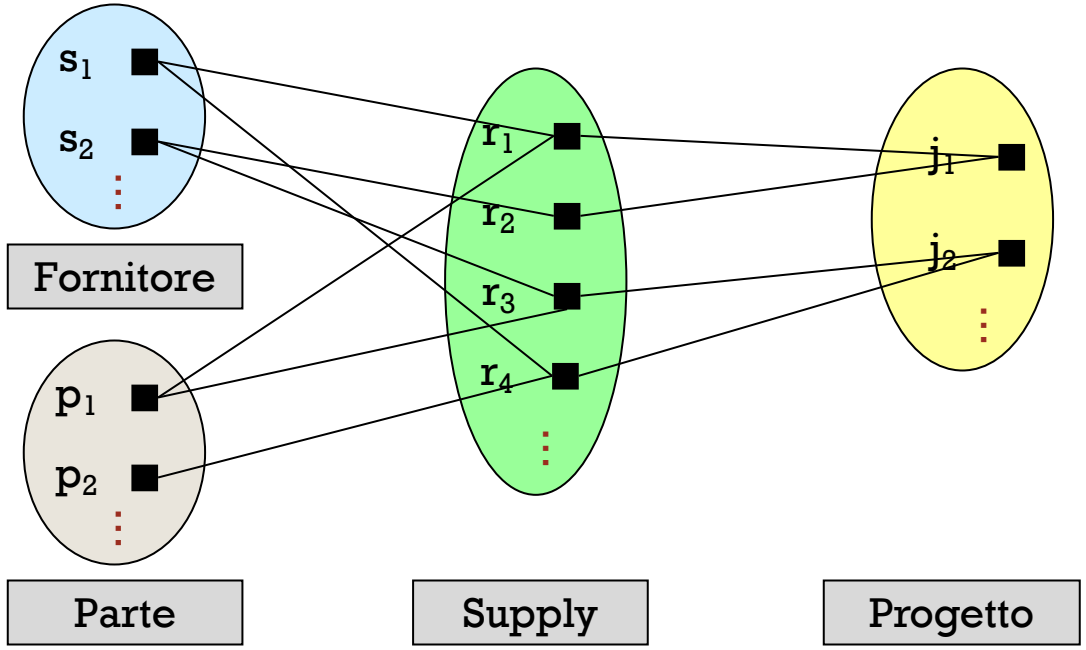
- **R** è un insieme di istanze di relazione r_i dove ogni r_i associa n entità $(e_1, e_2, \ldots, e_n)$, e ciascuna entità e_j in r_i è un membro del tipo di entità $\boldsymbol{E_j}$, con $1 \leq j \leq n$.
- Quindi un tipo di relazione è una funzione matematica su $\boldsymbol{E_1, E_2, \ldots E_n}$ o alternativamente può essere definita come un sottoinsieme del prodotto cartesiano $\boldsymbol{E_1 \times E_2 \times \ldots \times E_n}$.
- Ciascun $\boldsymbol{E_j}$ è detto *partecipante* al tipo di relazione **R.**
- Ogni entità individuale e_j è detta partecipante all'istanza di relazione

$$r_i = (e_1, e_2, \ldots, e_n).$$

- Il grado di un tipo di relazione è il numero di entità che vi partecipano.
  - Se il grado è 2, la relazione è detta binaria.

GRADO DI UN TIPO DI RELAZIONE

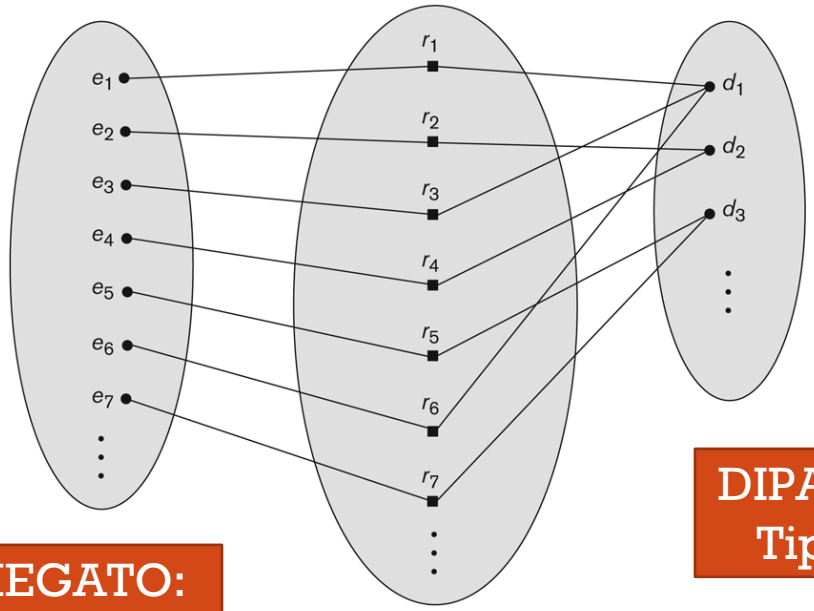
- Il grado di un tipo di relazione è il numero di tipi di entità partecipanti.
  - ***Es.:*** LAVORA_PER è di grado 2 (binaria).
  - ***Es.***: La relazione SUPPLY è un tipo di relazione ternaria, dove ogni istanza di relazione r_i associa tre entità, un fornitore *s*, una parte *p* e un progetto *j* ogni volta che *s* fornisce la parte *p* al progetto *j*.



Le relazioni possono essere di qualsiasi grado ma le più ricorrenti sono quelle binarie.

RELAZIONI COME ATTRIBUTI

- Può venire da pensare ad una relazione come ad un attributo di una determinata entità.
- Un IMPIEGATO può avere un attributo *Dipartimento*, il cui valore è un *riferimento* all'entità DIPARTIMENTO per cui lavora l'impiegato
  - L'insieme dei valori per l'attributo *Dipartimento* è l'insieme entità DIPARTIMENTO.
  - Analogamente possiamo inserire la lista di IMPIEGATI come attributo multivalore per ogni entità DIPARTIMENTO!
  - Se facciamo entrambi, siamo vincolati a mantenere gli attributi consistenti tra loro.
  - **Soluzione: RELAZIONI!**



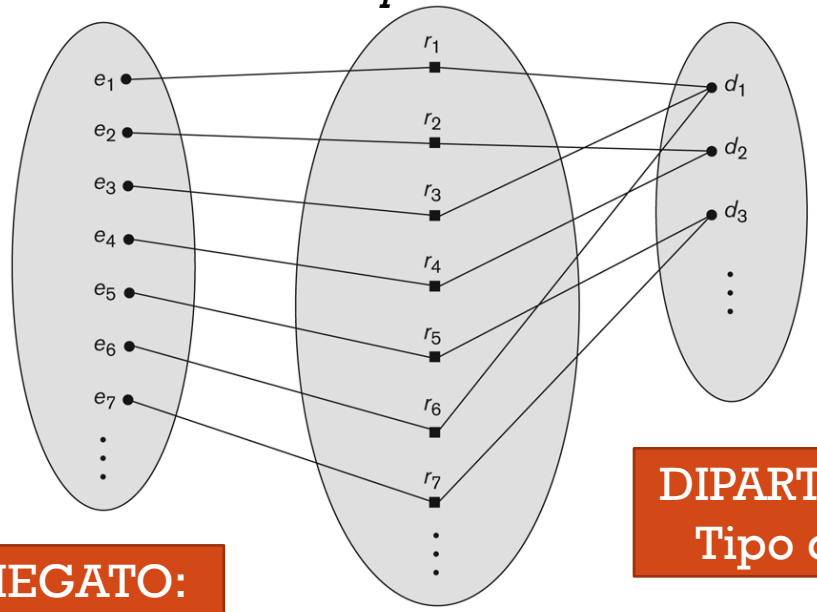
IMPIEGATO: Tipo di Entità

LAVORA_PER: Relazione

DIPARTIMENTO: Tipo di Entità

RUOLI E RELAZIONI RICORSIVE

- Ogni entità partecipa ad una relazione recitando un **ruolo** per quella relazione.
- Nella relazione LAVORA_PER:
  - l'IMPIEGATO ricopre il ruolo di *lavoratore* o di *impiegato*
  - Il *DIPARTIMENTO* ha il ruolo di *datore di lavoro* o di *dipartimento*

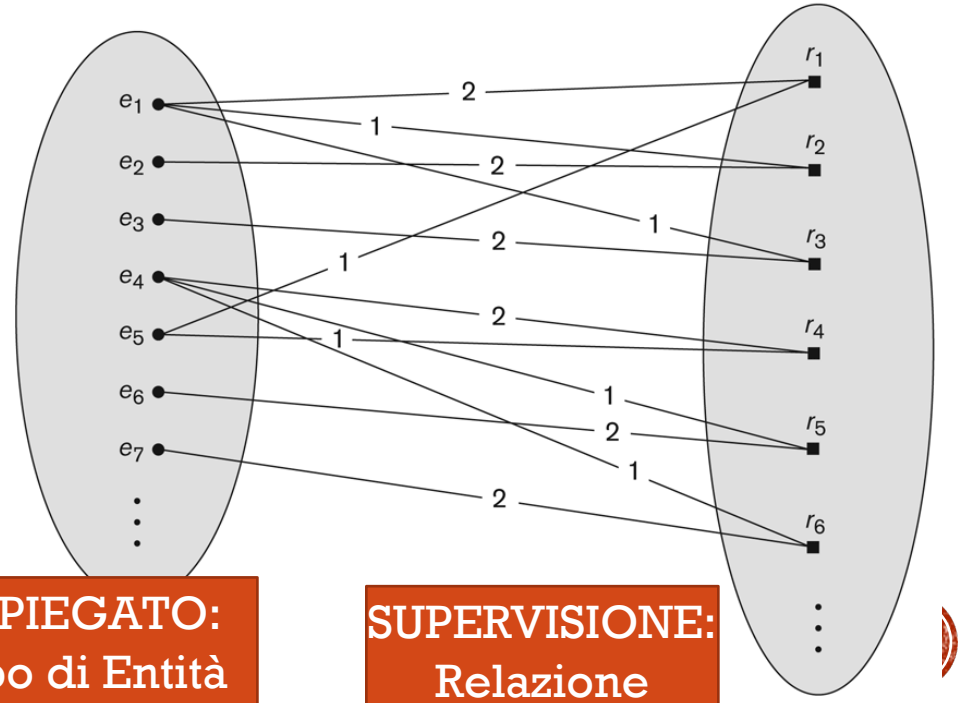


IMPIEGATO: Tipo di Entità

LAVORA_PER: Relazione

DIPARTIMENTO: Tipo di Entità

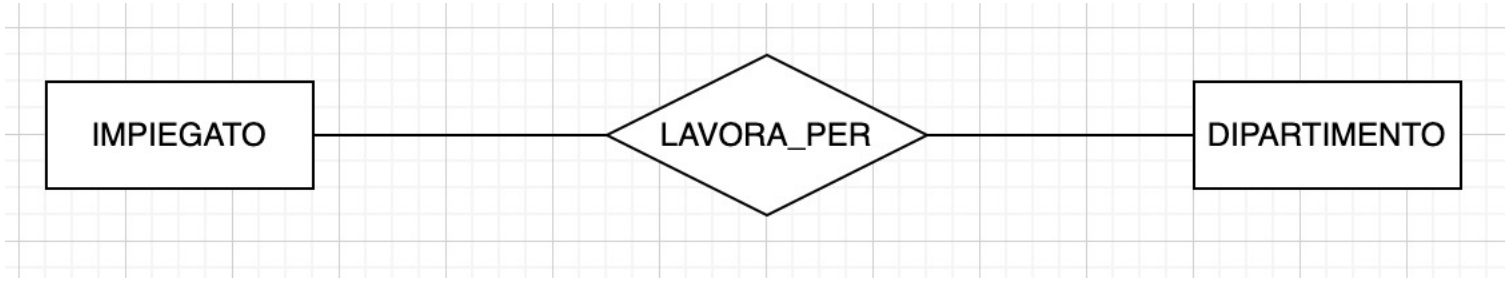
- Il nome di ruolo è importante per specificare il ruolo che ha un entità in una relazione.
- In alcuni casi un'entità partecipa più volte ad una relazione, ogni volta con ruoli diversi

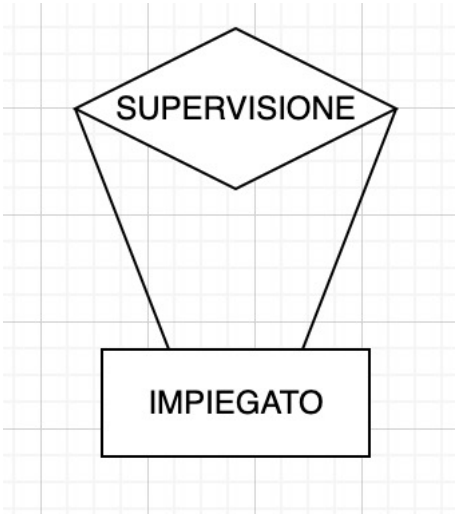


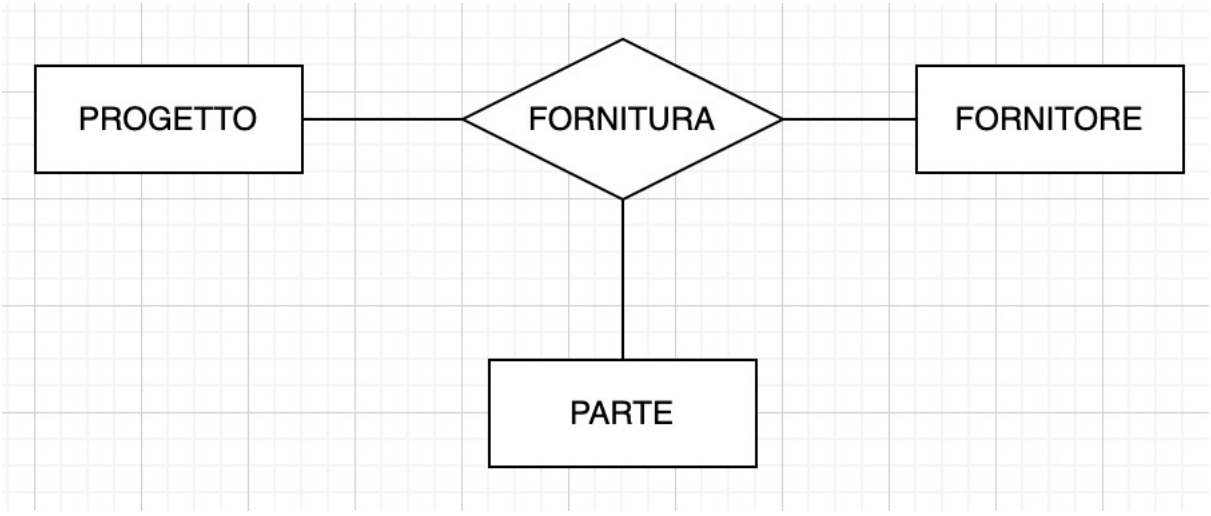
IMPIEGATO: Tipo di Entità

SUPERVISIONE: Relazione

RAPPRESENTAZIONE DI RELAZIONI (ER)







RELAZIONI E VINCOLI

- Le relazioni hanno dei vincoli che limitano le possibilità di combinazione tra le entità nella formazione di relazioni
- I vincoli sono determinati dal **MINIMONDO** che andiamo a rappresentare
- **Es.:** Un'azienda può avere i seguenti vincoli:
  - Un impiegato può lavorare per uno ed un solo dipartimento
  - Un impiegato può avere solo un supervisore
  - Un dipartimento deve avere almeno una sede
  - Un dipartimento deve avere solo un dirigente
- È possibile distinguere due tipi di vincoli di relazione:
  - Vincolo di cardinalità
  - Vincolo di partecipazione



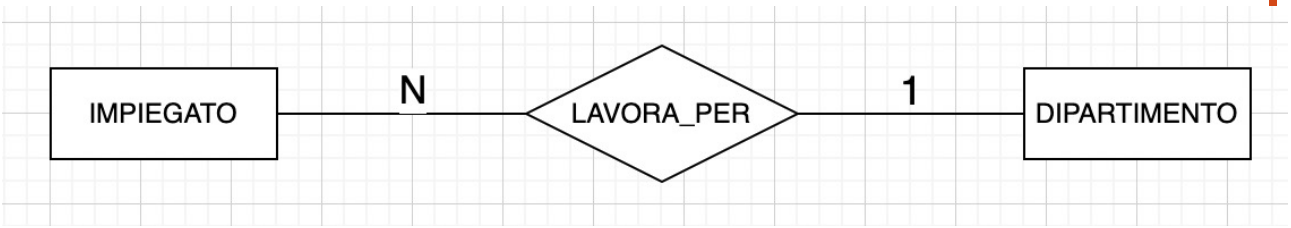
VINCOLI DI…

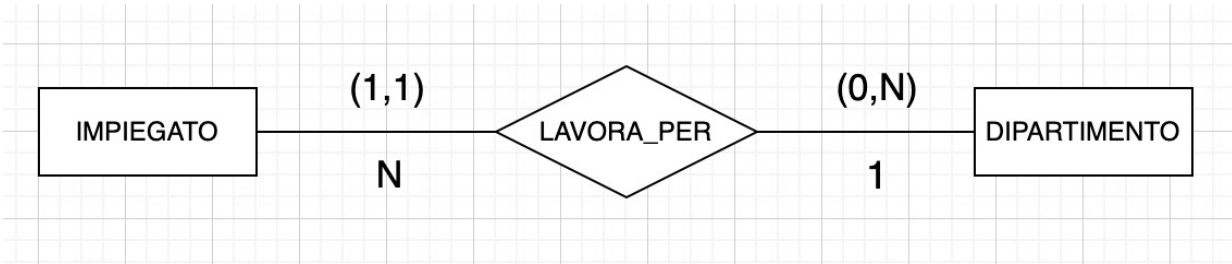
CARDINALITÀ

- Deve essere indicato per ciascun tipo di entità che partecipa ad una relazione, e permette di specificare il **numero massimo** di istanze di relazione a cui le occorrenze delle entità coinvolte possono partecipare.

PARTECIPAZIONE

- Serve per specificare se l'esistenza di un'entità dipende dalla sua partecipazione ad una relazione. Permette di specificare il **numero minimo** di istanze di relazione a cui le entità coinvolte possono partecipare





"Impiegato lavora per Dipartimento", con rapporto di cardinalità N:1 –
N impiegati lavorano per un dipartimento.

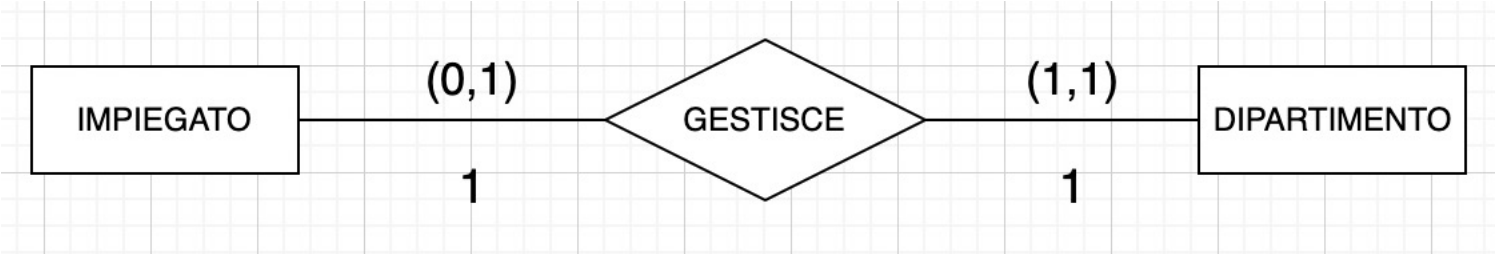
MAX: Ogni dipartimento può avere numerosi impiegati, e ciascun impiegato lavora per un solo dipartimento.
MIN: Ogni impiegato deve afferire ad almeno un dipartimento, ma un dipartimento può anche non avere impiegati

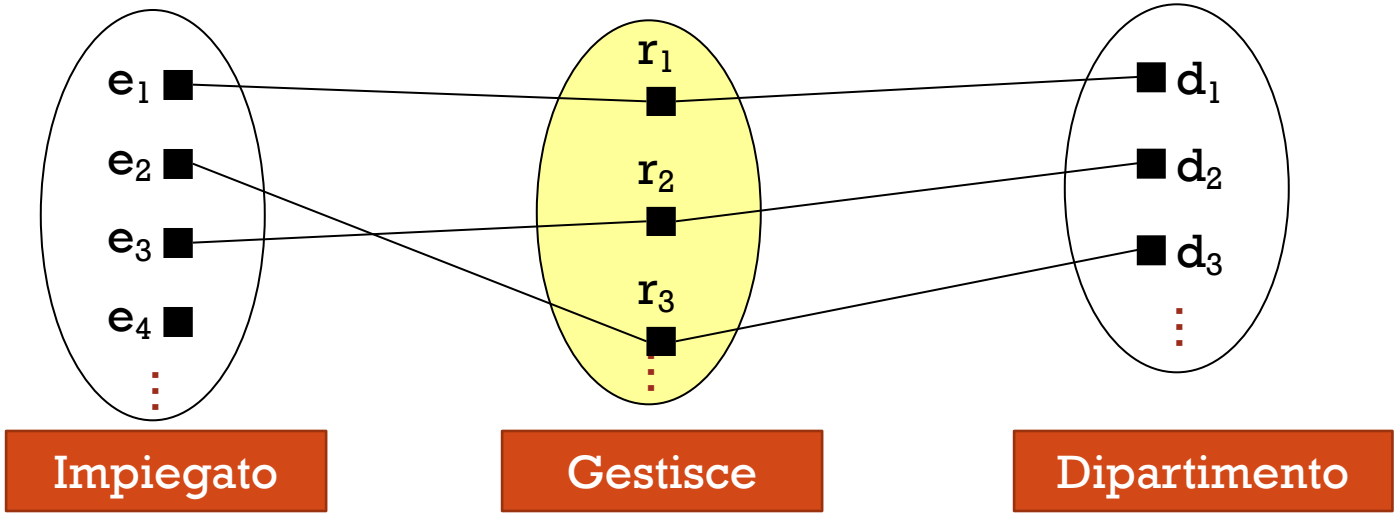
VINCOLI DI CARDINALITA' E PARTECIPAZIONE

- È possibile assegnare un qualunque intero non negativo a un rapporto di cardinalità, con l'ovvio vincolo che la cardinalità minima deve essere minore o uguale alla cardinalità massima.
- Nella maggior parte dei casi si utilizzano solo tre valori: 0, 1 e N:
  - Il valore 0 per la cardinalità minima indica una partecipazione opzionale del tipo di entità alla relazione.
  - Il valore 1 per la cardinalità minima indica una partecipazione obbligatoria del tipo di entità alla relazione.
- La cardinalità minima può eventualmente essere omessa, quella massima deve essere sempre specificata.

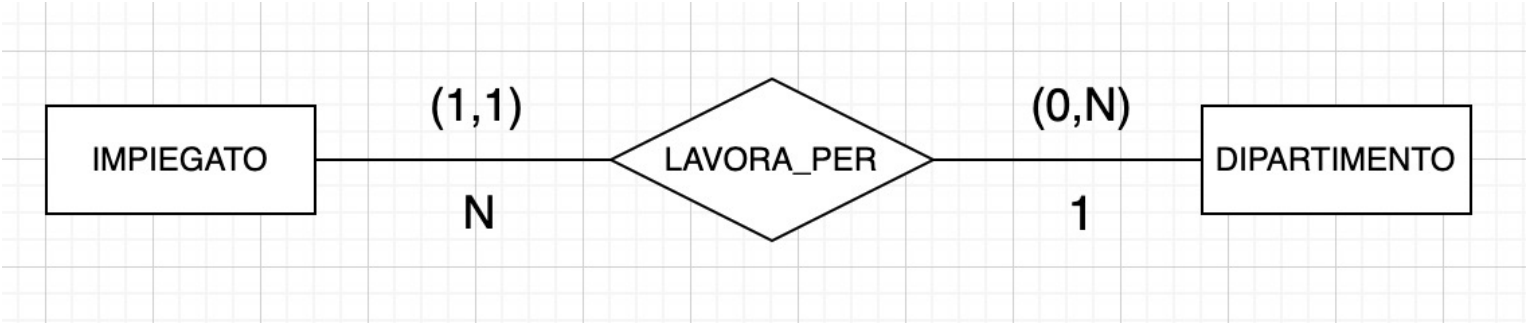


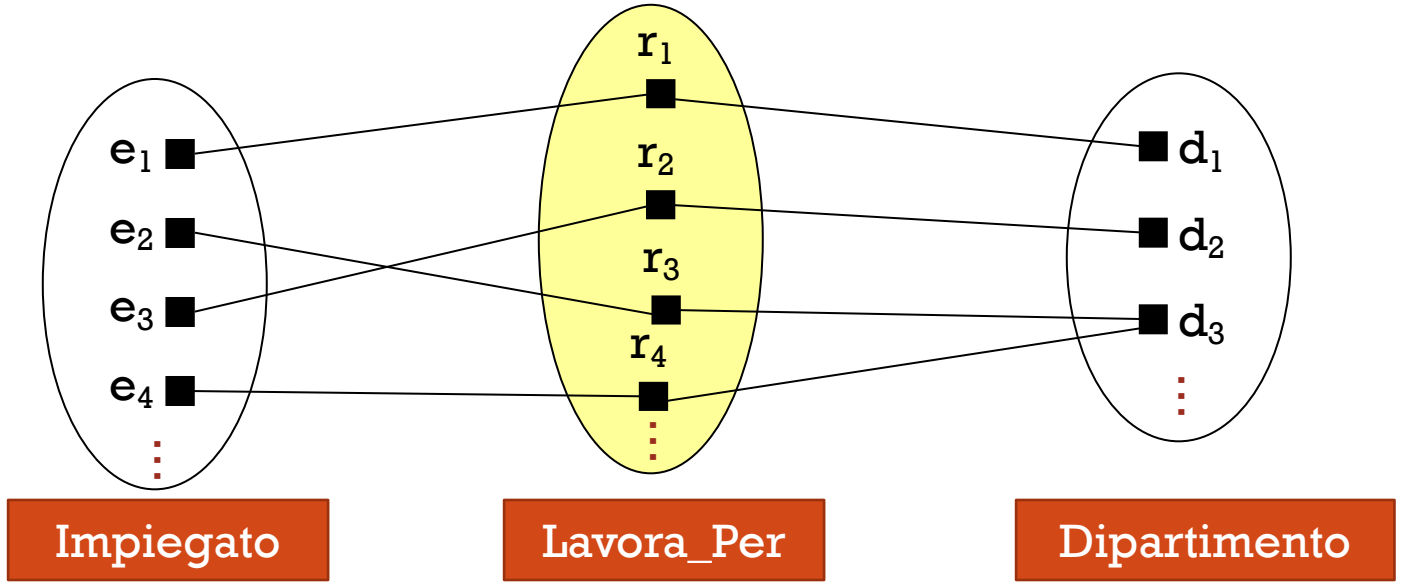
RELAZIONI 1:1



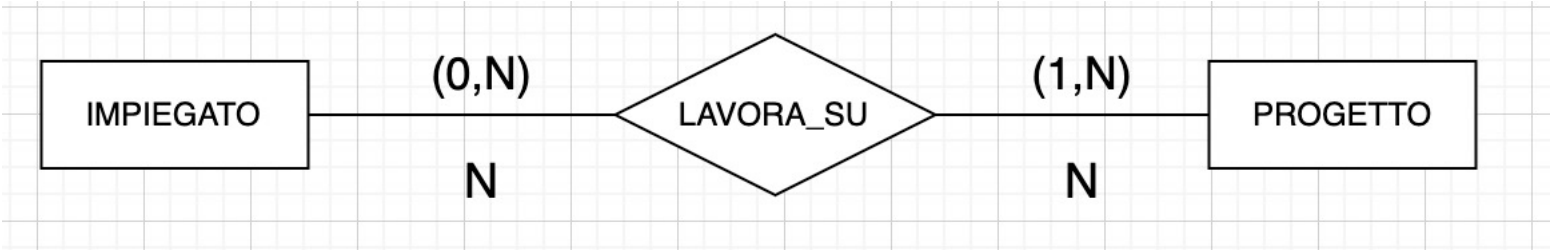


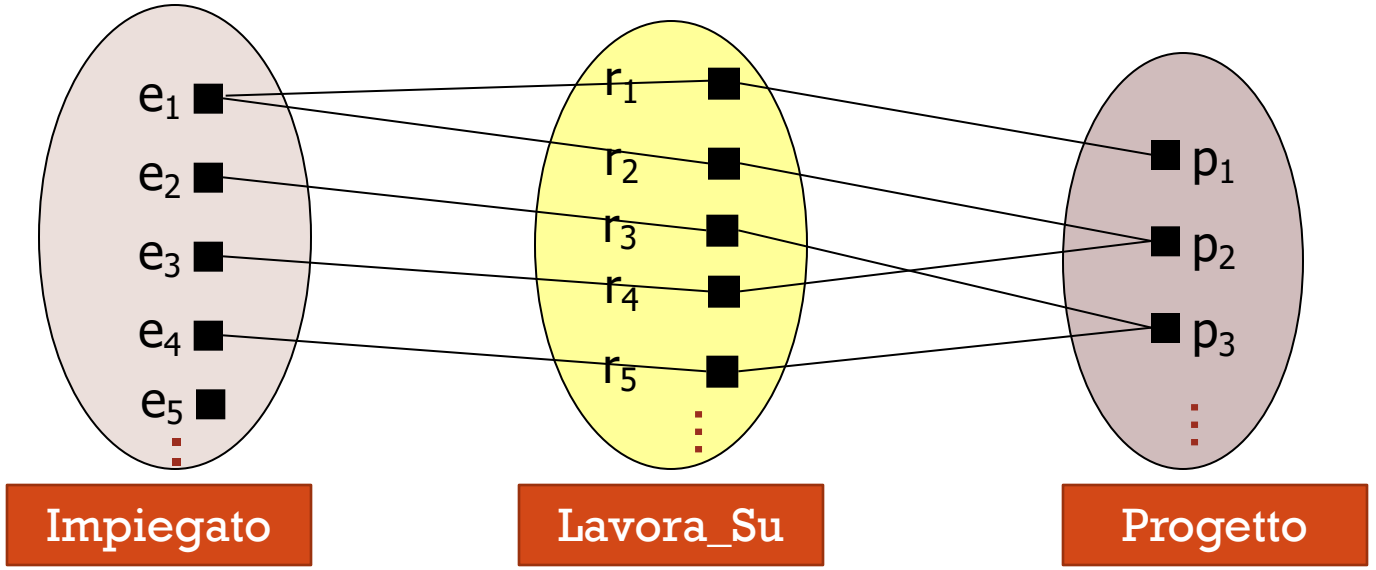
RELAZIONI 1:N (N:1)





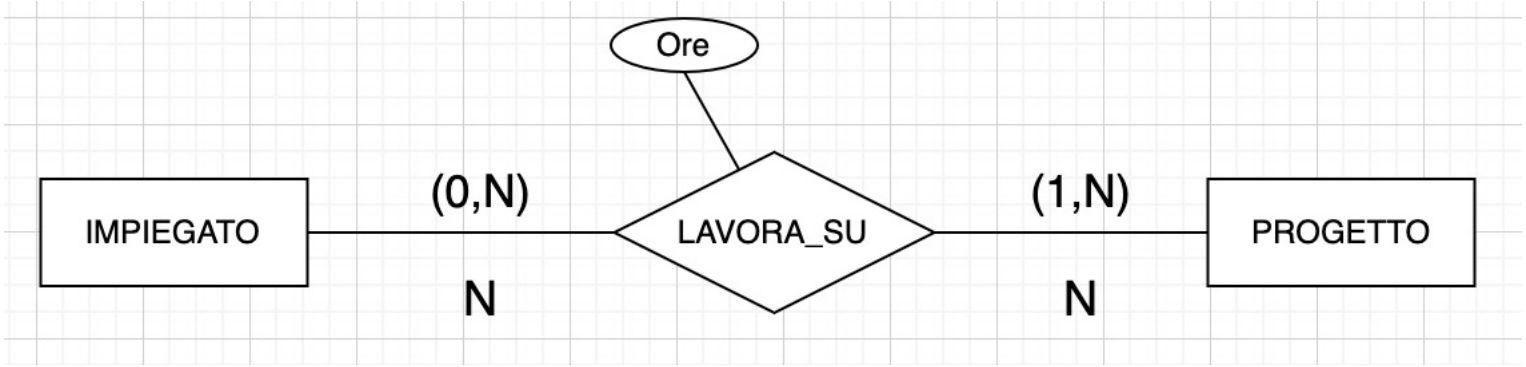
RELAZIONI N:N

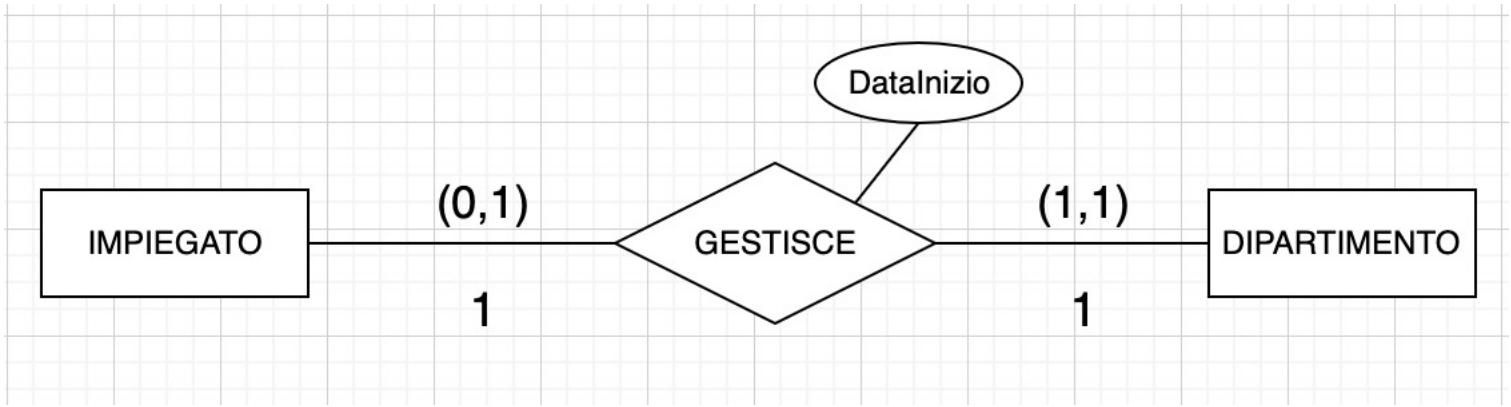




ATTRIBUTI DI RELAZIONI

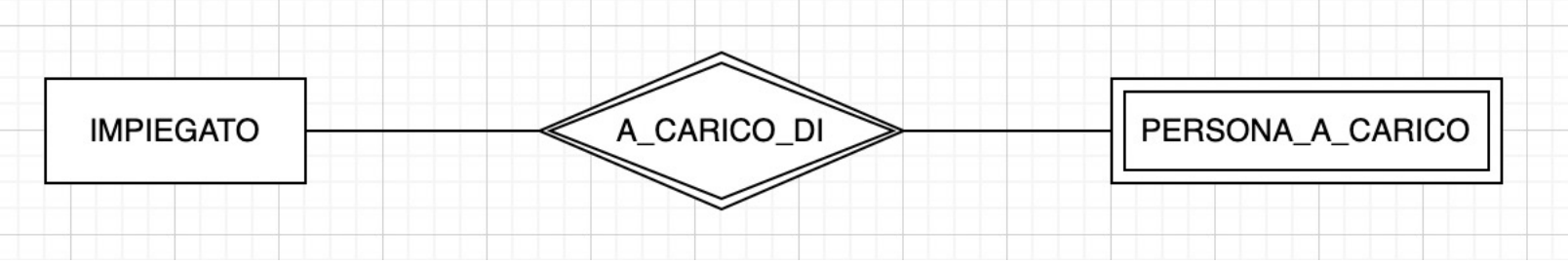
- Anche le relazioni, come le entità, possono avere degli attributi che le descrivono





ENTITÀ DEBOLI

- Le **entità deboli** sono entità che non possiedono veri e propri attributi chiave
  - Quelle viste fino ad ora sono appunto dette **entità forti**
- Tali entità sono identificate tramite il loro collegamento con entità di un altro tipo
  - **Entità identificante** o **proprietario**
- La relazione che correla l'entità debole all'entità forte è detta **relazione identificante**
- Un tipo di entità debole ha solitamente una **chiave parziale**, ossia l'insieme degli attributi che identificano le entità deboli collegate alla stessa entità proprietaria.
  - Se non esiste, nel caso peggiore si avrà che tutti gli attributi dell'entità debole faranno parte della chiave parziale.

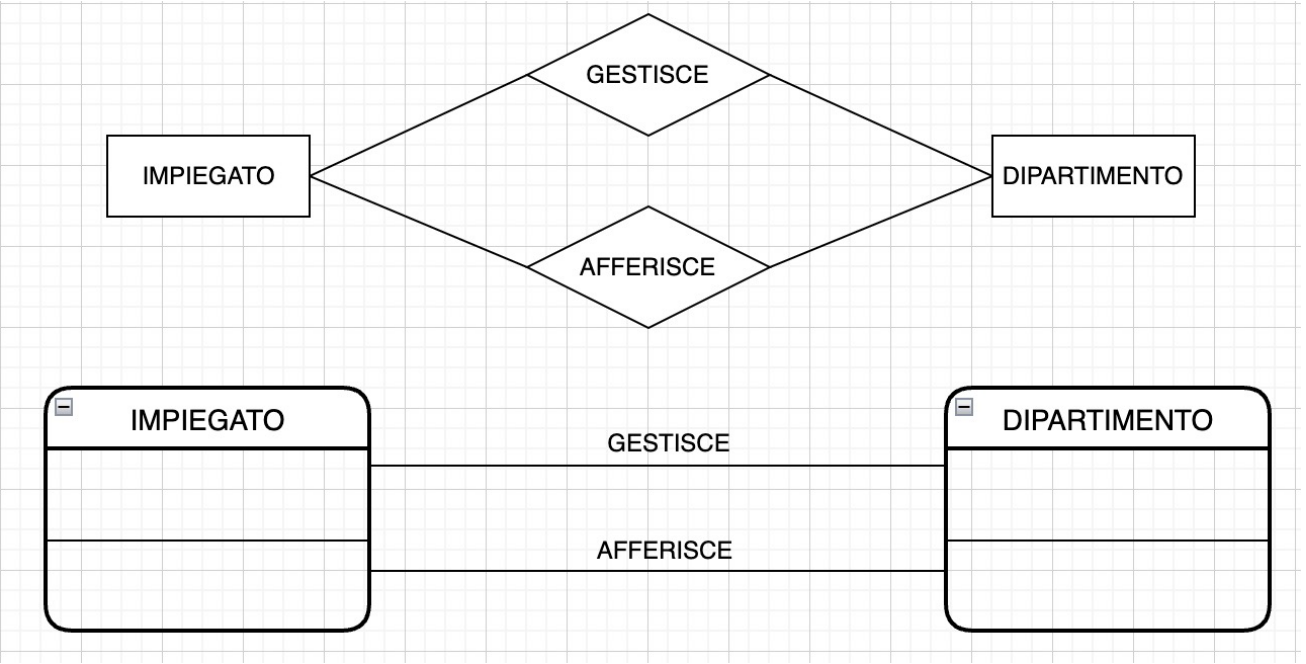


LE ASSOCIAZIONI (UML)



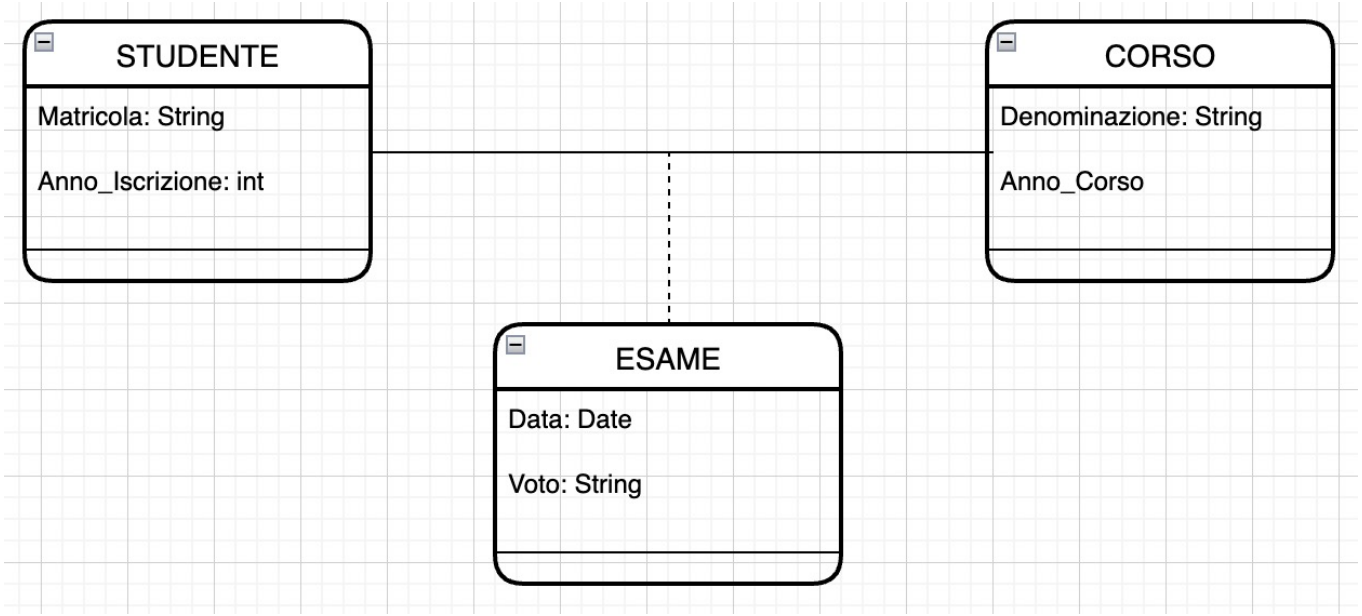
LE ASSOCIAZIONI

- **Associazioni:** Corrispondono alle relazioni del modello ER.
  - Le associazioni binarie si rappresentano con semplici linee che congiungono le classi coinvolte.
  - Il nome della relazione viene posto sulla linea.
    - *Non è obbligatorio:* infatti, in UML possono esistere relazioni senza nome.



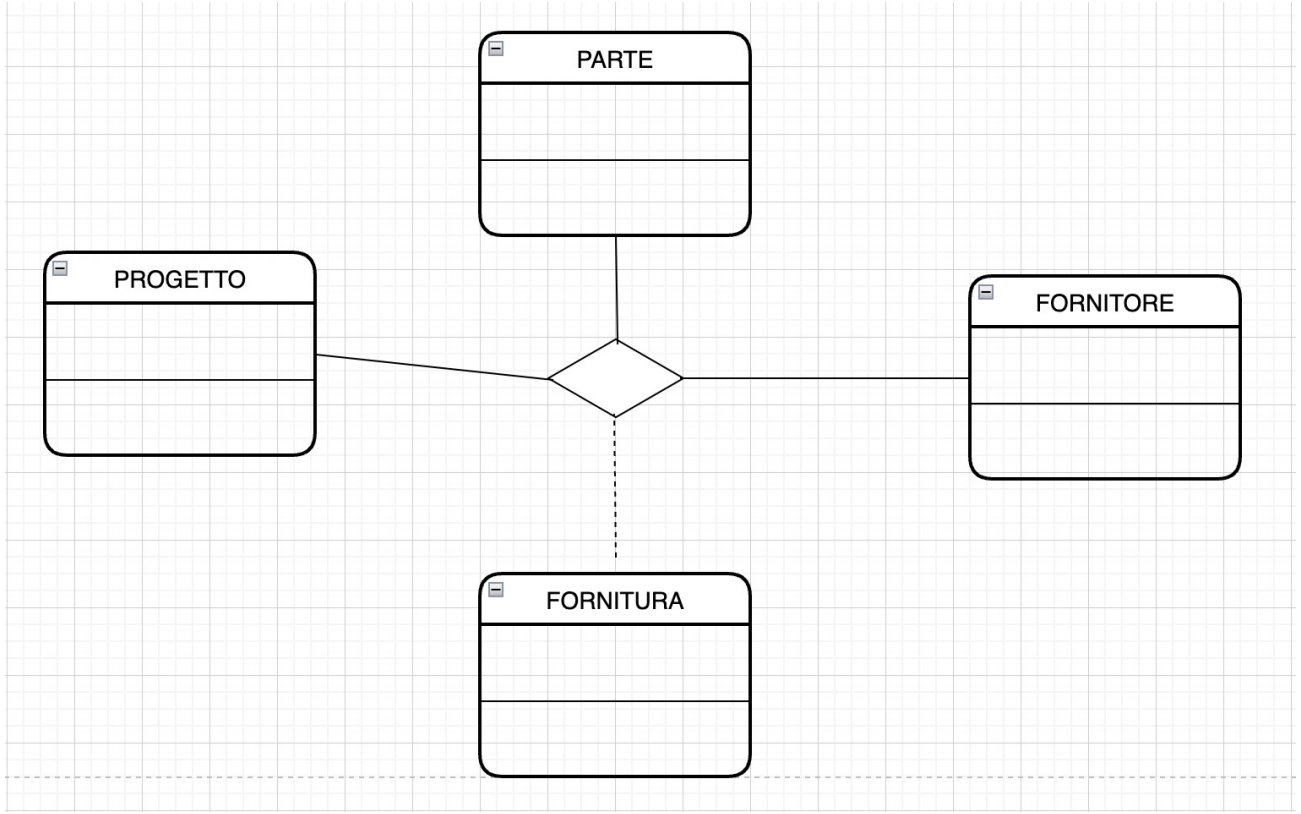
CLASSI DI ASSOCIAZIONE

- Per assegnare attributi ad una associazione si fa uso delle *classi di associazione* per descrivere le proprietà di una associazione.
  - Queste classi vengono collegate all'associazione da descrivere mediante una linea tratteggiata.



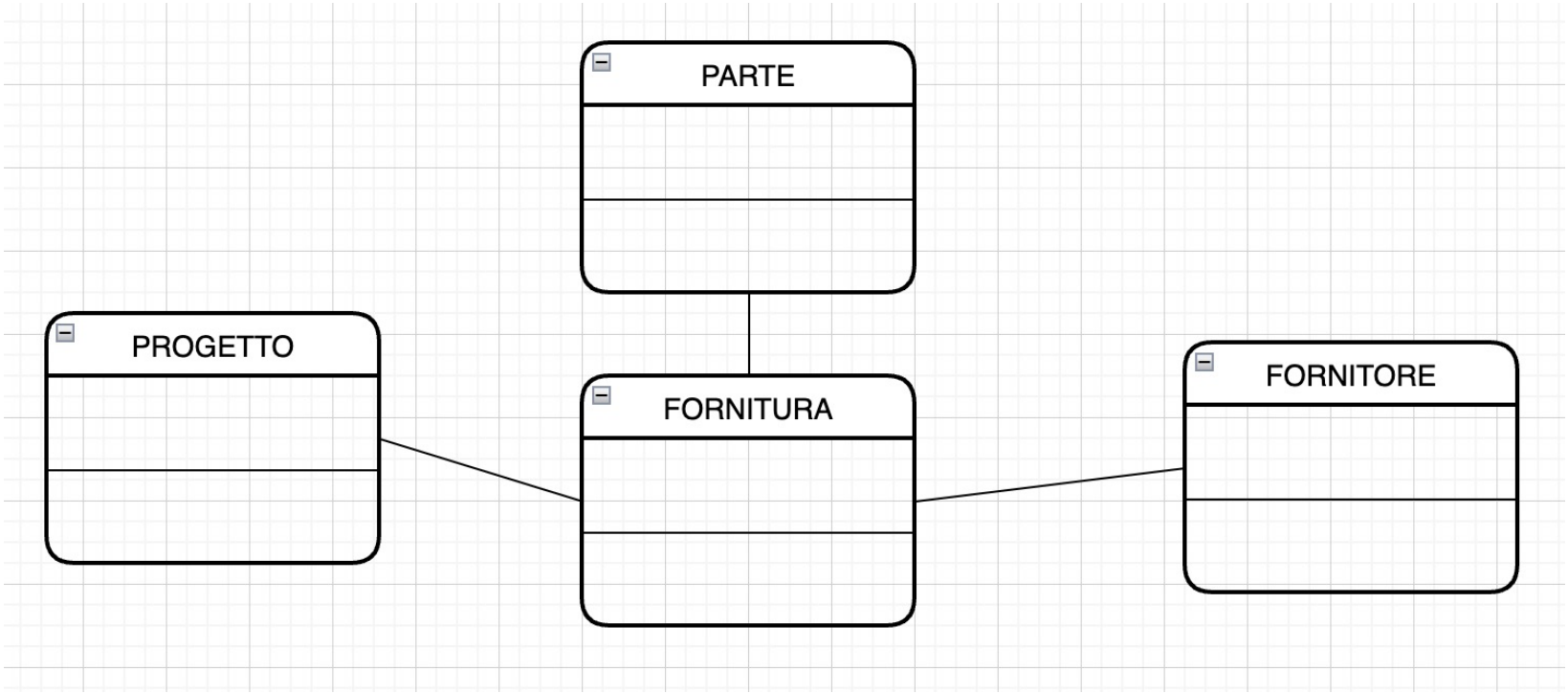
ASSOCIAZIONI N-ARIE

- L'associazione viene rappresentata da un *rombo* collegato con le classi che partecipano all'associazione.
  - Si può usare una *classe di associazione* per assegnare attributi all'associazione n-aria.



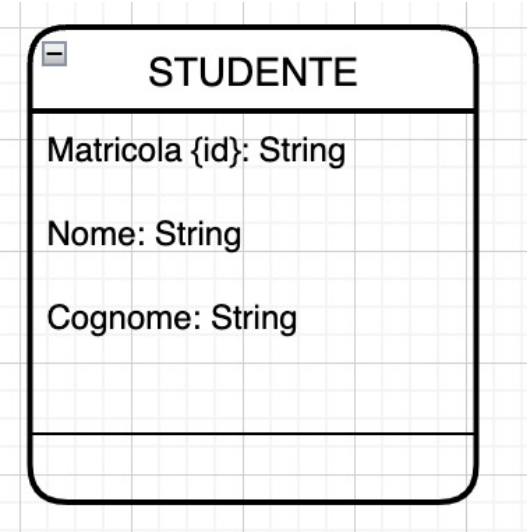
ASSOCIAZIONI N-ARIE (2)

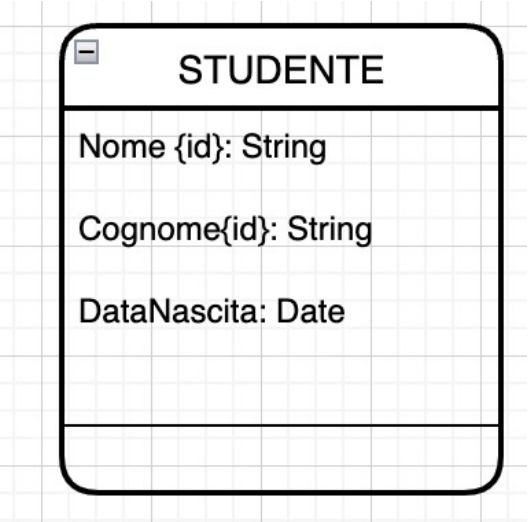
- L'uso di associazioni n-arie è raro nel diagramma delle classi.
  - Si può applicare un processo di *"reificazione"*: ovvero trasformare l'associazione in una classe legata alle altre tramite associazioni binarie.



IDENTIFICATORI (INTERNI)

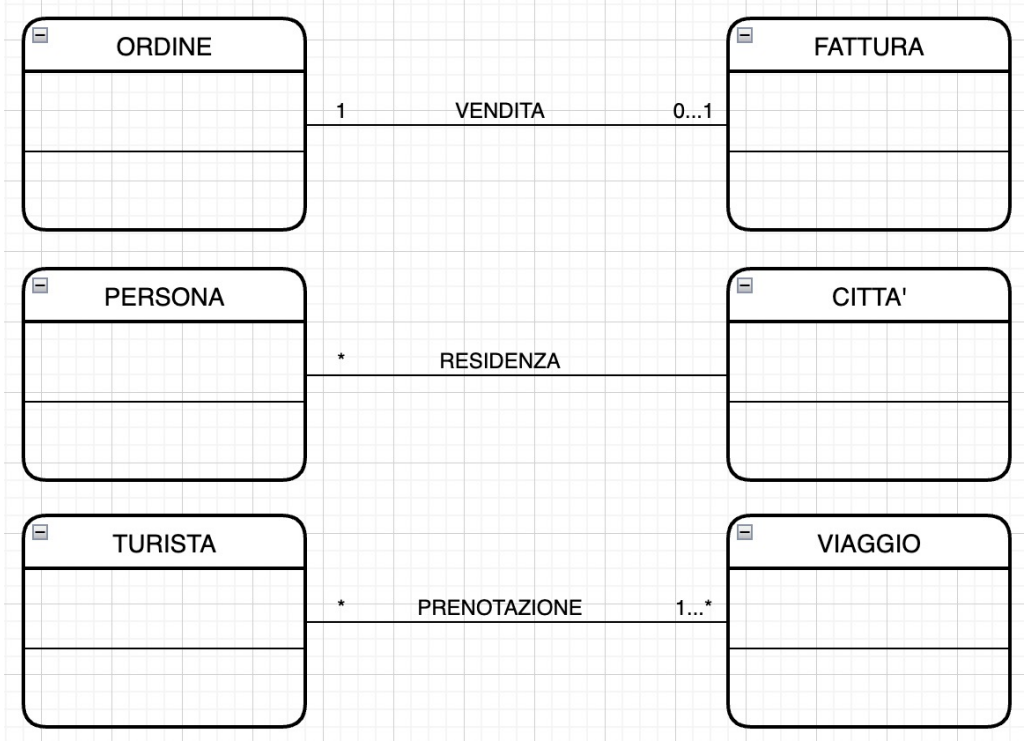
- In UML non esiste una notazione per esprimere identificatori di classi.
- Si possono definire vincoli di integrità su associazioni e attributi specificandoli tra parentesi graffe (*vincolo utente*).
  - ***Es.*** {*id*} indica che l'attributo è un identificatore.
  - ***Es.*** assegnare {*id*} a più attributi indica un identificatore composto.



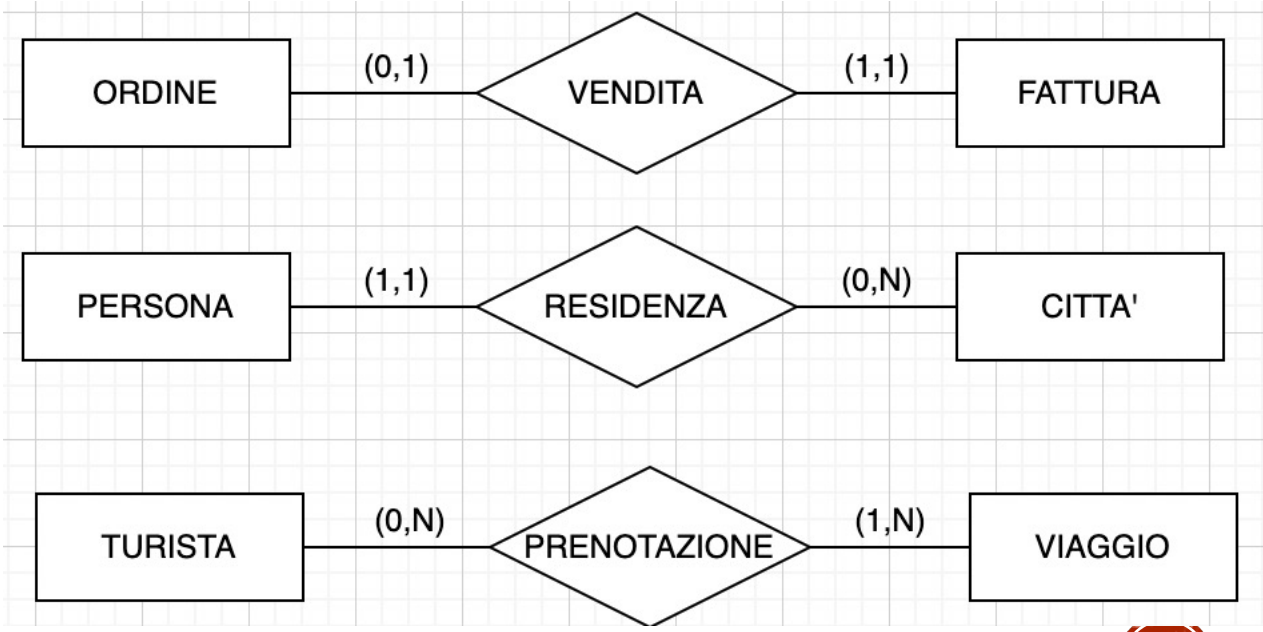


ASSOCIAZIONI CON MOLTEPLICITÀ

- La molteplicità di default è 1 (che denota la coppia di cardinalità 1..1).
  - Le cardinalità di default possono essere omesse.
- La cardinalità "*molti*" viene rappresentata dal simbolo * (dove si intende 0..*, ovvero (0, N) dello schema ER).

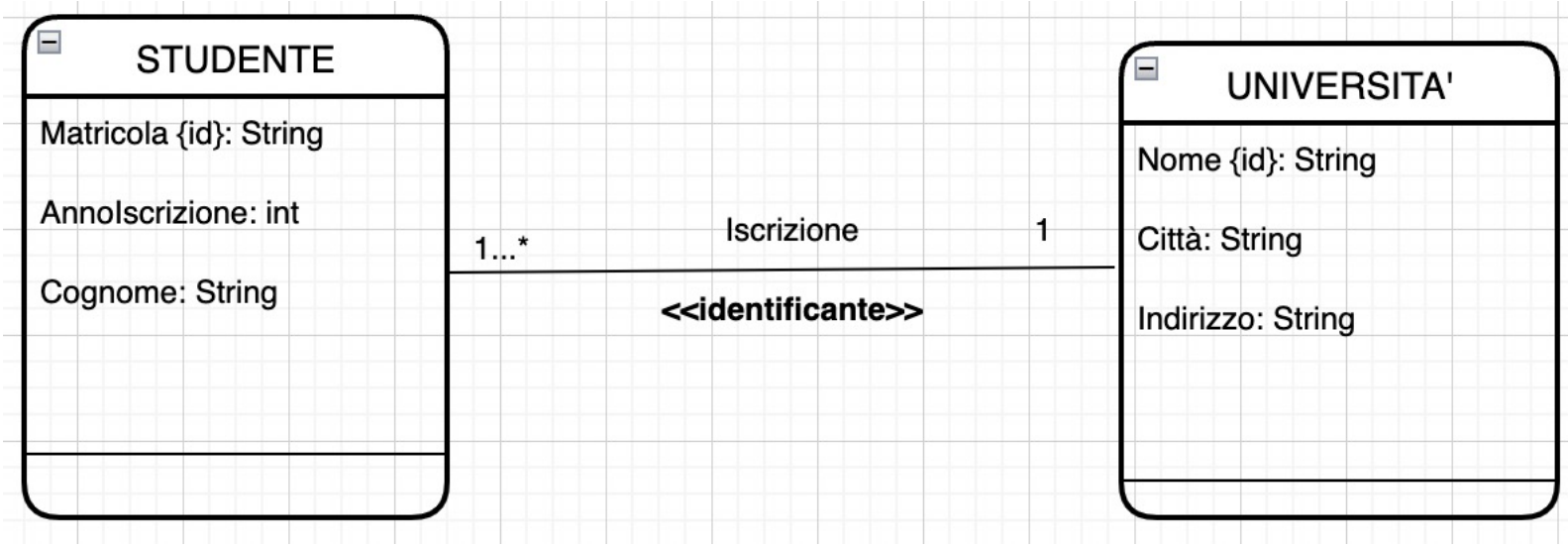


Corrispondenza (invertita) con le molteplicità dell'ER:



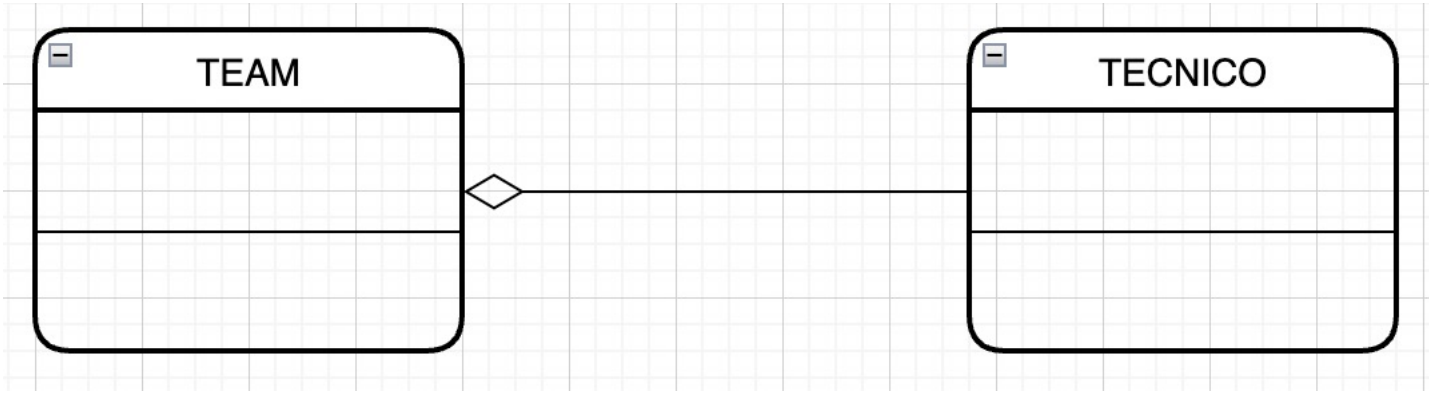
IDENTIFICATORE ESTERNO

- Per gli identificatori esterni si usa uno *stereotipo*.
  - Si usano per estendere i costrutti base dell'UML:
    - quando si vuole modellare un concetto che non può essere modellato con i costrutti di base.
  - Vengono indicati da un nome racchiuso tra i simboli << e >>.
  - ***Es.*** Lo stereotipo <<*identificante*>> insieme alla *Matricola* identifica univocamente uno *Studente* rispetto una particolare *Università*.
  - In tal caso Studente è un'entità debole e Matricola è la sua chiave parziale



PROPRIETÀ DELLE ASSOCIAZIONI

- Con una freccia è possibile indicare un verso privilegiato di *navigabilità* di un'associazione.
- Si possono specificare *aggregazioni* di concetti: relazioni tra un oggetto composito e uno o più concetti che ne costituiscono una sua parte.
  - Sono indicate da linee avente un rombo dal lato del concetto "*aggregante*".

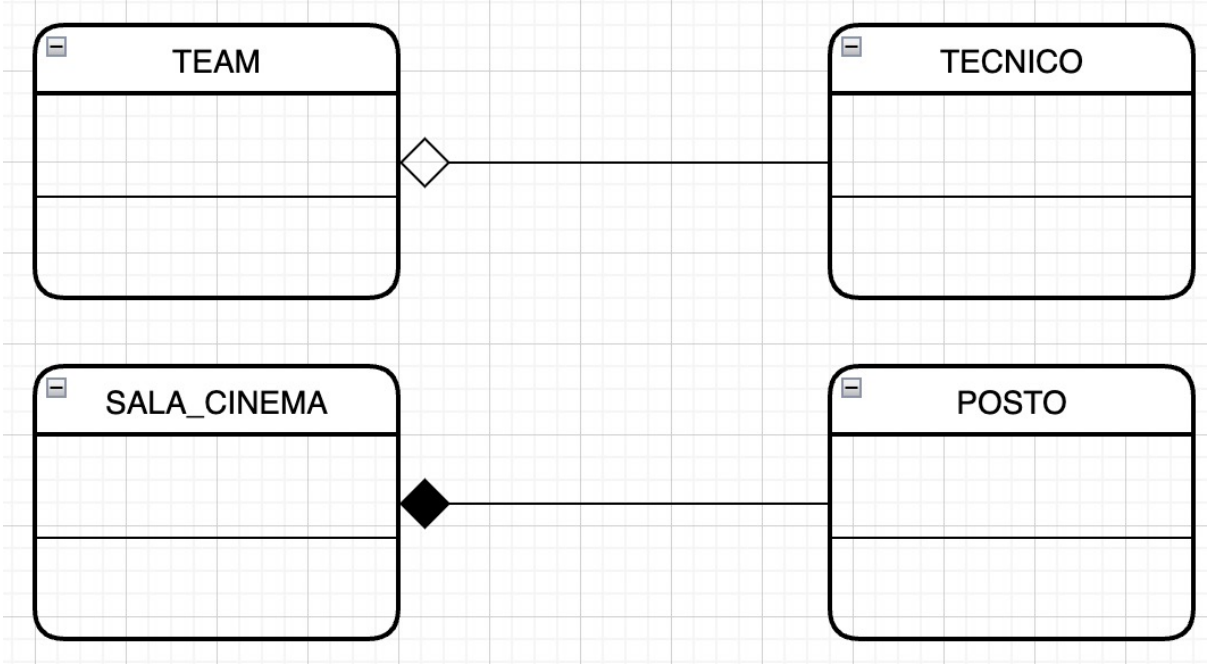


aggregante

parte

PROPRIETÀ DELLE ASSOCIAZIONI (2)

- Il rombo bianco indica che un oggetto della classe "*parte*" può esistere senza dover appartenere a un oggetto della classe "*aggregante*". **(Aggregazione)**
- Il rombo nero indica bianco indica che un oggetto della classe "*parte*" **non** può esistere senza appartenere a un oggetto della classe "*aggregante*". **(Composizione)**

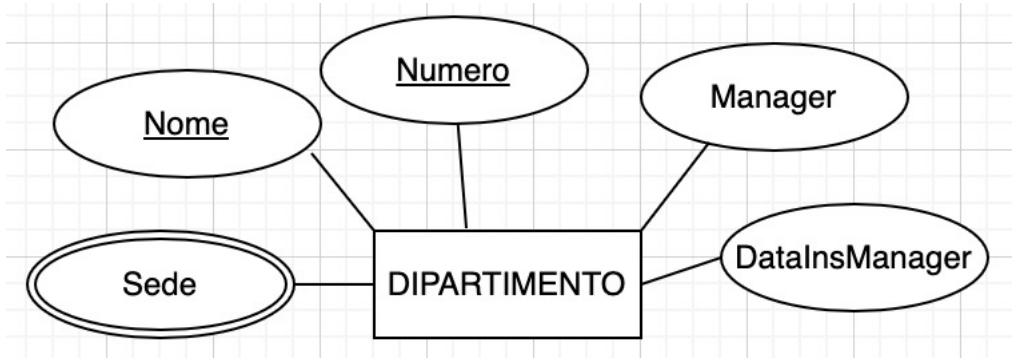


RAFFINAMENTO DEL DB COMPANY



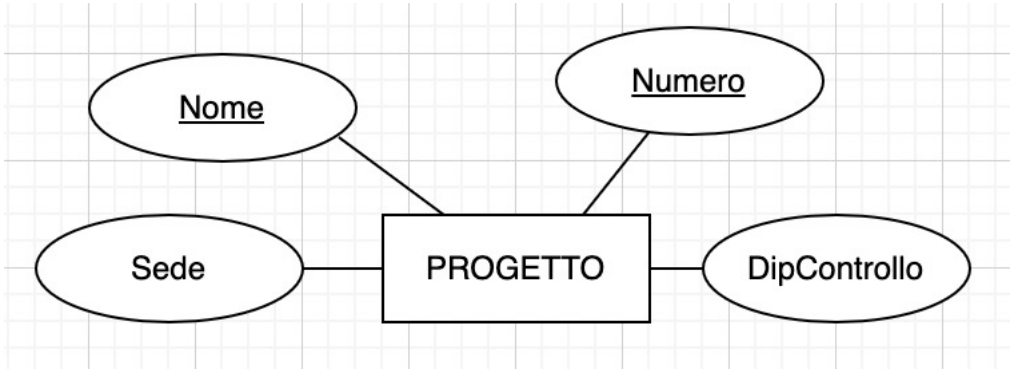
DIPARTIMENTI

- La compagnia è organizzata in DIPARTIMENTI. Ogni Dipartimento ha un nome, un numero ed un impiegato che lo gestisce. Bisogna tener traccia della data di insediamento del manager. Un dipartimento può avere più locazioni.



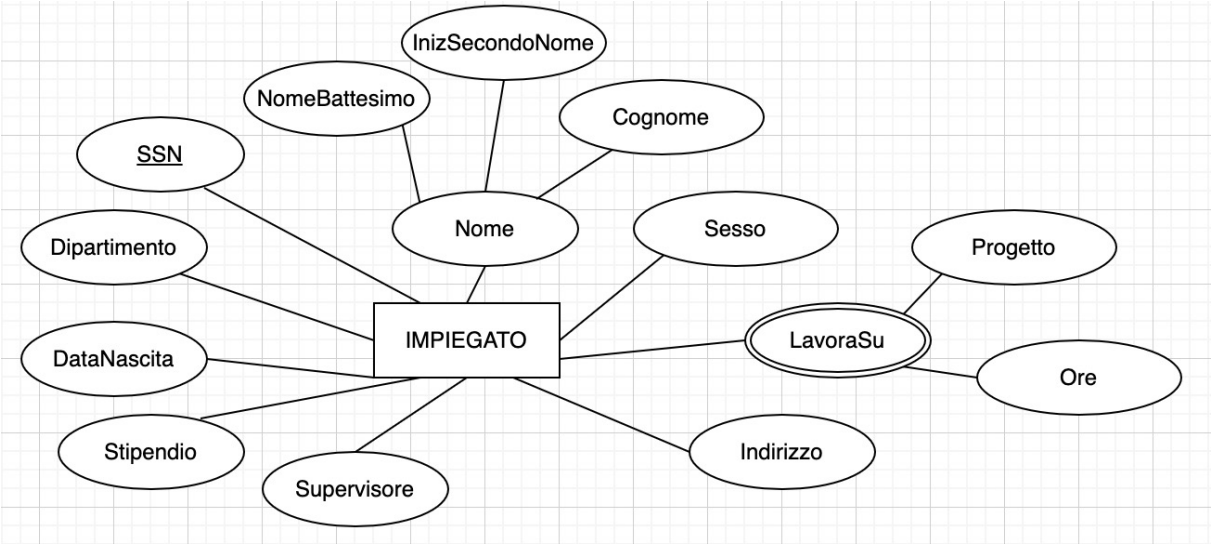
PROGETTI

- Ogni dipartimento controlla una serie di PROGETTI. Ogni progetto ha un nome, un numero ed una singola locazione.



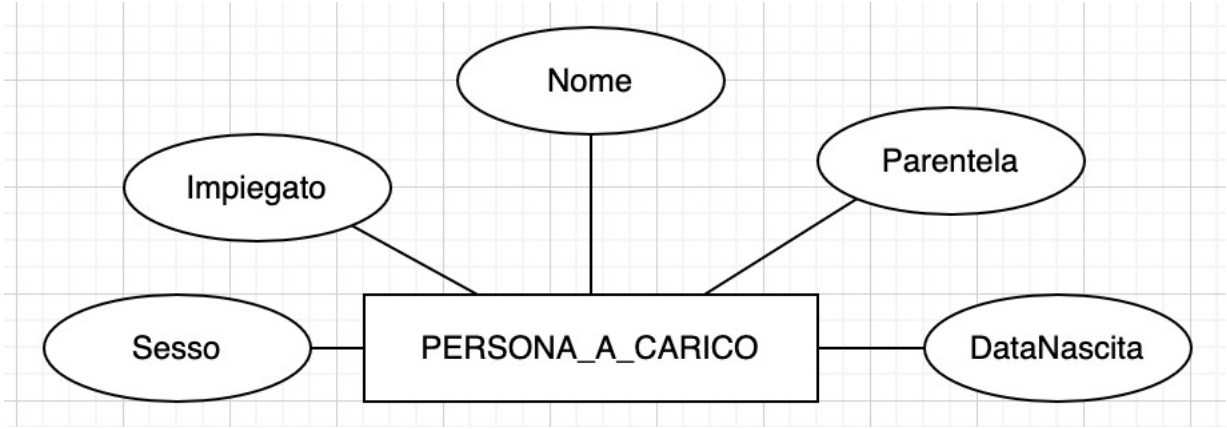
IMPIEGATI

- Per IMPIEGATO si tiene traccia di: nome, SSN, indirizzo, salario, sesso e data di nascita. Ogni impiegato lavora per un dipartimento e può lavorare su più progetti. Teniamo traccia del numero di ore settimanali che un impiegato spende su un progetto e del supervisore di ogni impiegato.



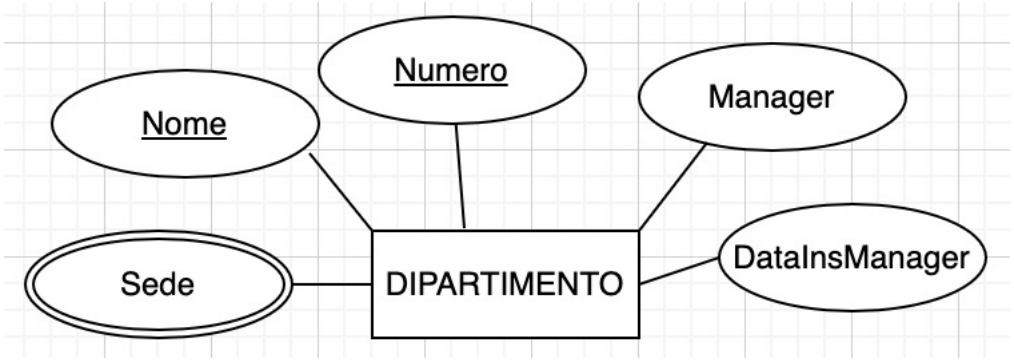
PERSONE A CARICO

- Ogni impiegato ha una serie di PERSONE A CARICO. Per ogni persona a carico, registriamo: nome, sesso, data di nascita e parentela con l'impiegato.



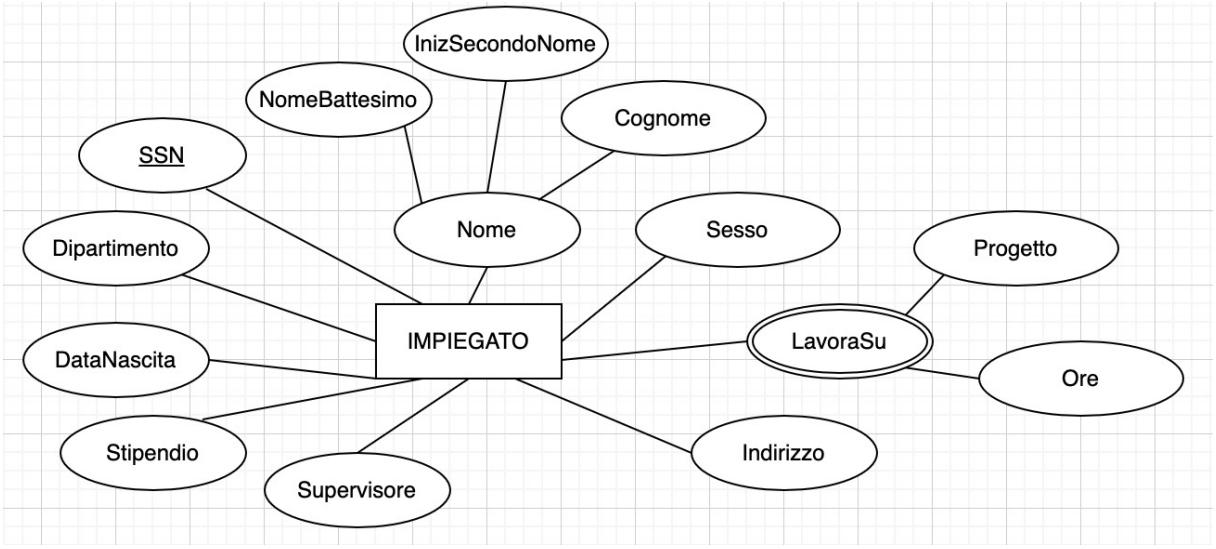
RELAZIONI IDENTIFICATE - DIPARTIMENTO

- L'attributo Manager è un riferimento ad un altro IMPIEGATO.
- L'attributo DataInsManager si riferisce alla data in cui un IMPIEGATO diventa manager di un DIPARTIMENTO
  - Può essere definito come attributo della relazione tra IMPIEGATO e DIPARTIMENTO



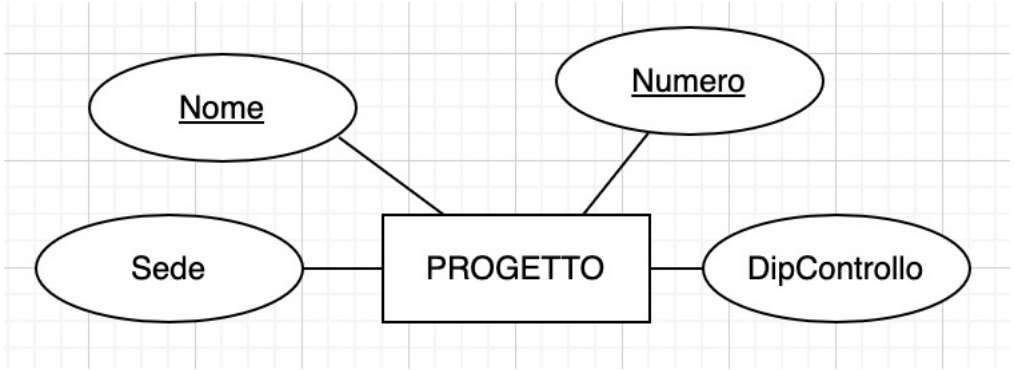
RELAZIONI IDENTIFICATE - IMPIEGATO

- L'attributo Supervisore è un riferimento ad un altro IMPIEGATO.
- L'attributo complesso LavoraSu consiste di
  - Un attributo Progetto che si riferisce ad un altro PROGETTO
  - Un attributo Ore che si riferisce alla relazione tra IMPIEGATO e PROGETTO
- L'attributo Dipartimento si riferisce al DIPARTIMENTO di afferenza dell'IMPIEGATO



RELAZIONI IDENTIFICATE - PROGETTO

- L'attributo DipControllo è un riferimento ad un DIPARTIMENTO.



RELAZIONI IDENTIFICATE – PERSONA_A_CARICO

- L'attributo Impiegato è un riferimento ad un IMPIEGATO.

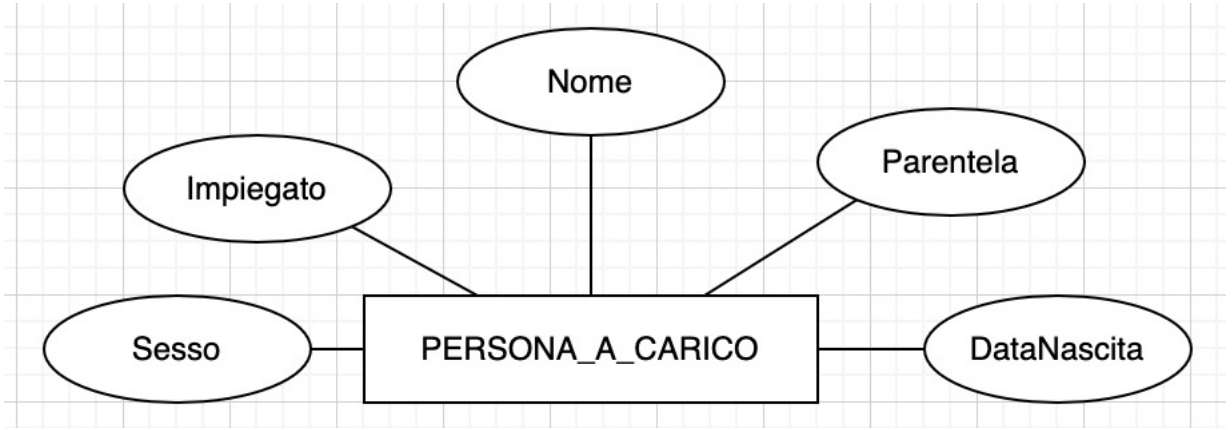
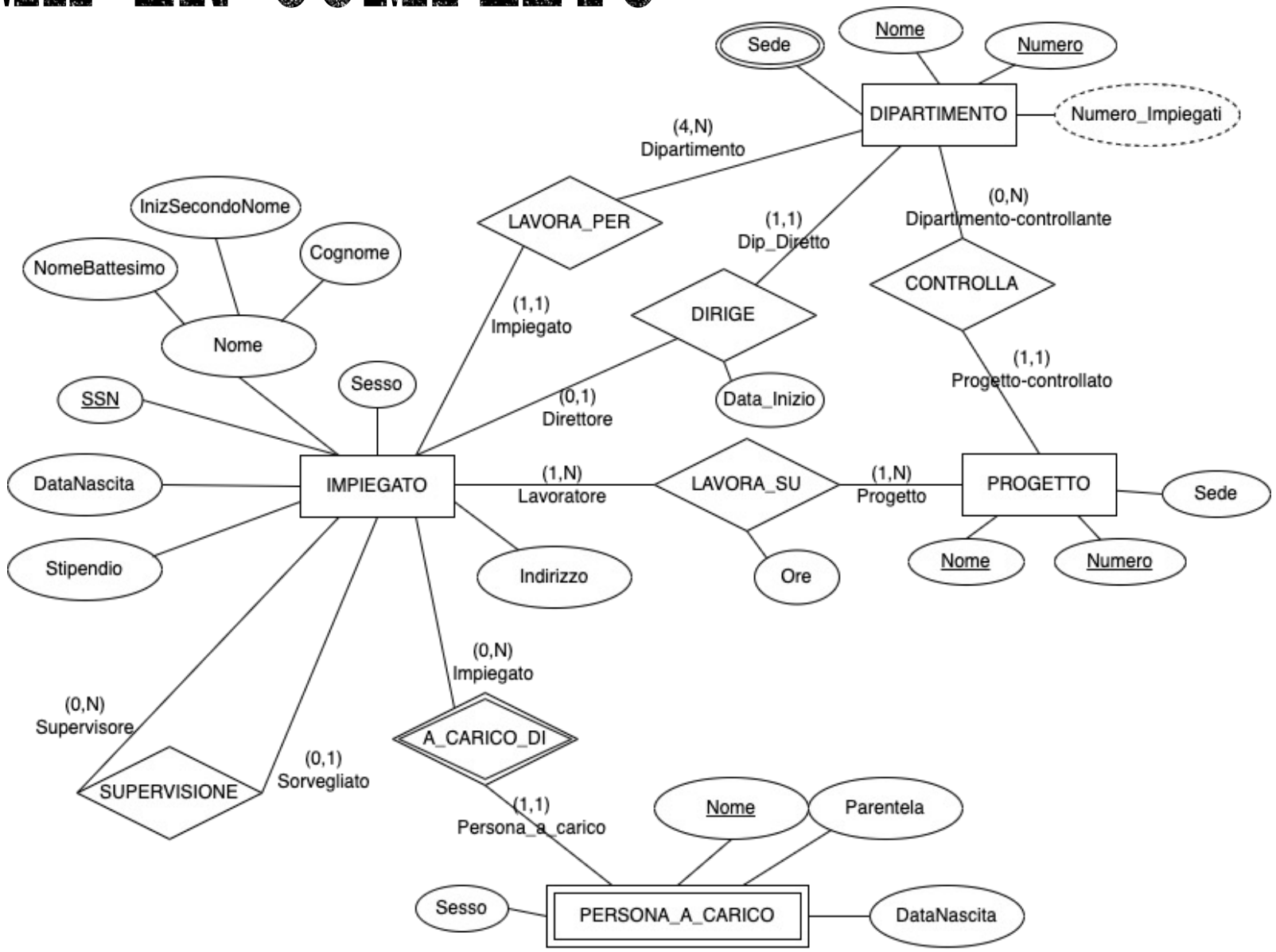
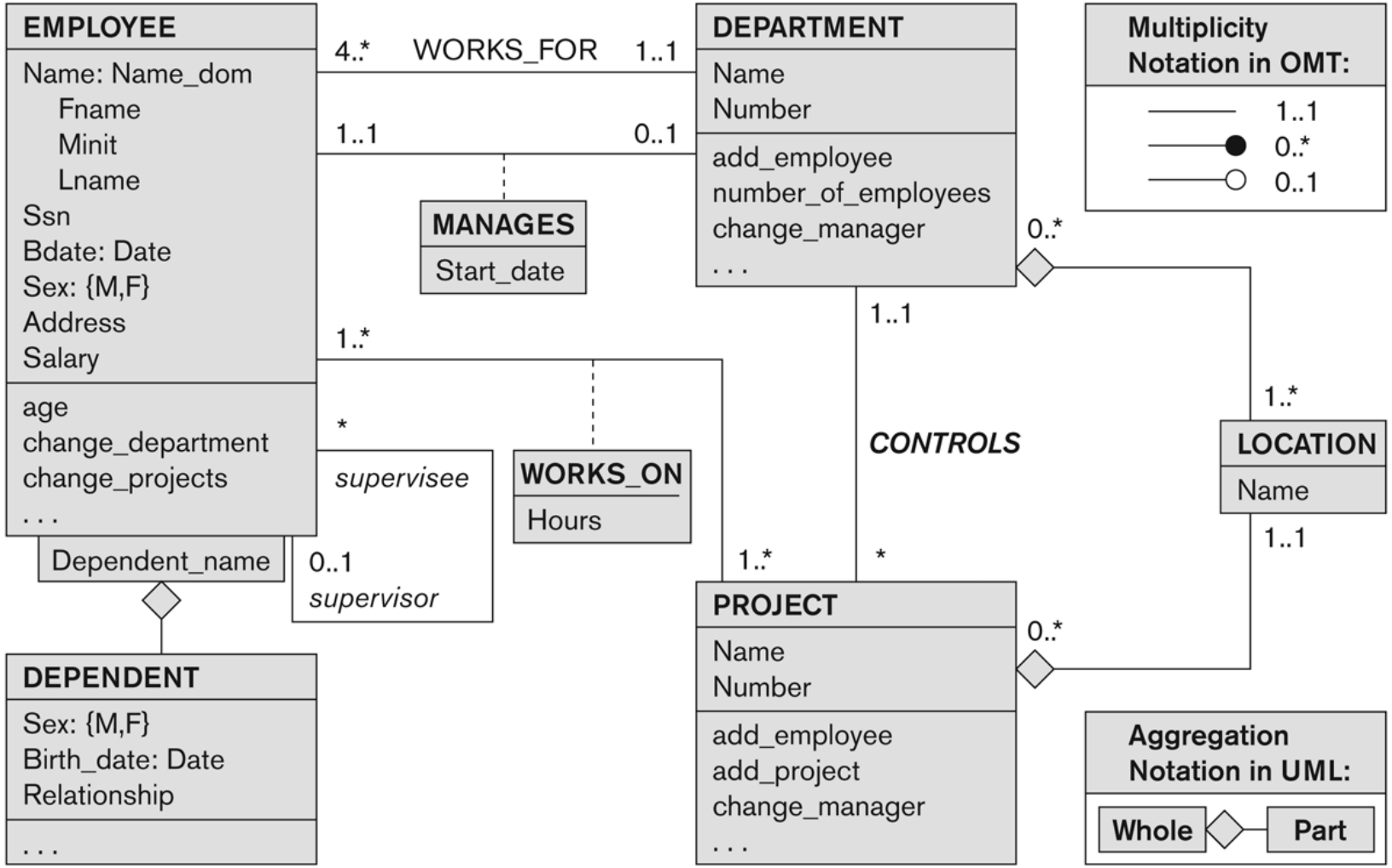


DIAGRAMMA ER COMPLETO



LO SCHEMA CONCETTUALE IN UML



LE NOTE

- Un diagramma delle classi può essere documentato con delle *note*.
  - Consistono in semplici commenti testuali.
  - Sono riportate sul diagramma in un rettangolo con l'angolo superiore destro ripiegato.
  - Una nota può essere associata ad un particolare elemento del diagramma attraverso una linea tratteggiata.



ESERCIZIO 1

- *Modellare uno schema riguardanti le informazioni di un campionato di calcio:*
  - L'entità **SQUADRA** rappresenta tutte le squadre del campionato, indicando per ognuna di esse il nome, la città e il nome dell'allenatore.
  - L'entità **GIOCATORE** rappresenta i giocatori delle squadre: ogni giocatore ha un contratto con una sola squadra e ogni squadra ha più giocatori. I giocatori sono identificati dal loro Codice Fiscale e per ognuno di essi è indicato il nome, il cognome, il ruolo nella squadra, la città di nascita e la data di nascita.
  - Lo schema contiene anche informazioni sulle partite. Una **PARTITA** è identificata con un numero (che deve essere differente per tutte le partite dello stesso giorno) e con un riferimento al giorno (attraverso la relazione COLLOCAZIONE e l'entità giornata).
  - Le relazioni CASA e OSPITE rappresentano le due squadre che giocano la partita: per ogni partita è indicato il risultato e l'**ARBITRO**, con la relazione ARBITRAGGIO tra partita e arbitro; questa entità rappresenta tutti gli arbitri del campionato e per ognuno di essi è indicato il Nome, il Cognome, la Città e la Regione. Un arbitro è rappresentato solo se ha arbitrato almeno una partita.
  - Una partita può essere giocata su campo neutrale o può essere rinviata ad un'altra data (ma questi due eventi non sono ammessi contemporaneamente nello schema).
  - La relazione Partecipazione rappresenta il fatto che un giocatore abbia giocato in una partita, la sua posizione (che può essere diversa dalla sua solita). Lo schema non esprime la condizione che i giocatori che giocano una partita devono avere un contratto con una delle due squadre.
  - L'entità **GIORNATA** rappresenta la giornata del campionato. Sono identificate con Numero e Girone. La relazione Posizione dà il punteggio di ogni squadra in ogni giornata.



MODELLAZIONE EER



MODELLO ENHANCED ER

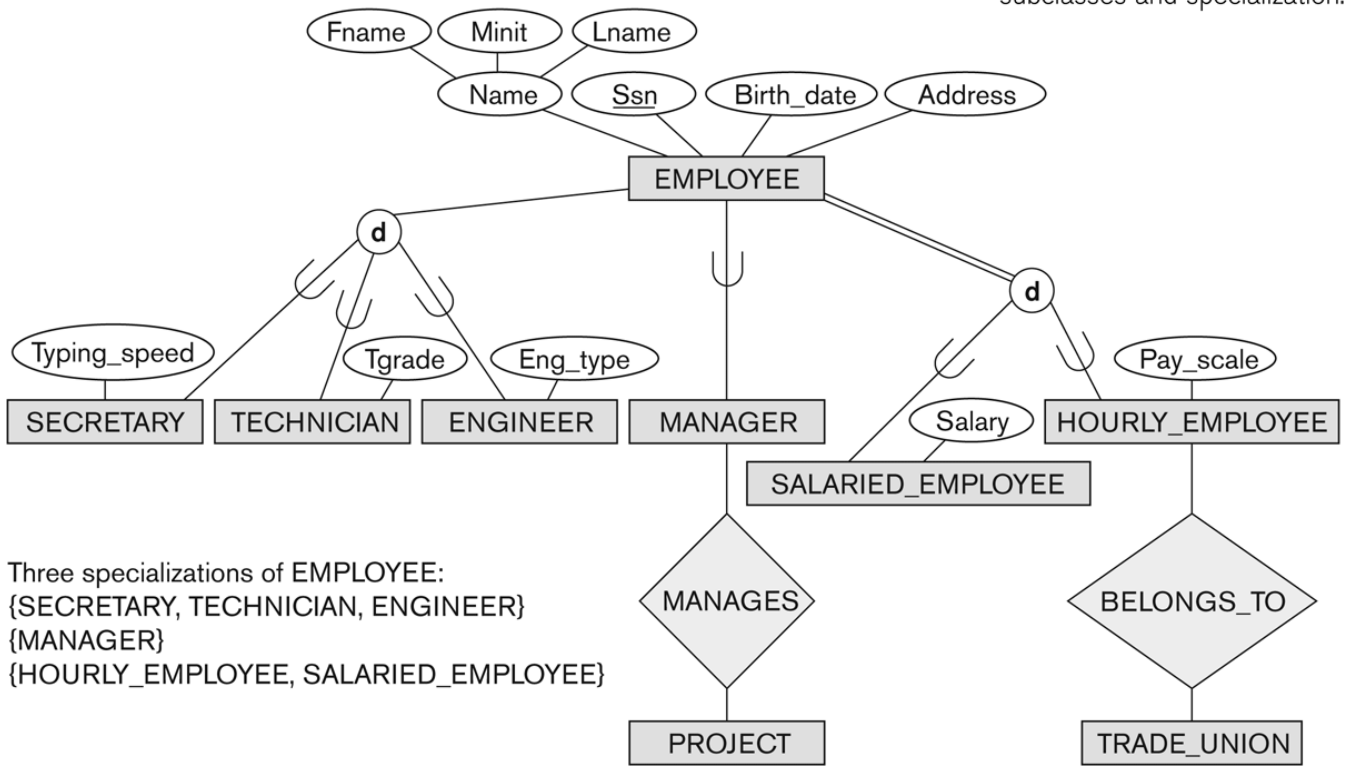
- Il modello EER offre dei costrutti per ampliare i concetti base dell'ER
  - Sottoclassi e Superclassi
  - Specializzazioni e Generalizzazioni
  - Categorie
  - …
- Questi nuovi strumenti servono per modellare un miniworld in maniera ancora più dettagliata
- Le notazioni utilizzate possono essere comprese da un set di utenti più specializzato
  - Alcuni concetti dell'EER provengono dai concetti di programmazione ad oggetti
    - Appunto le superclassi, le sottoclassi ed i relativi concetti di ereditarietà



SOTTOCLASSI, SUPERCLASSI ED EREDITARIETÀ

- Un entità può avere ulteriori estensioni per massimizzare la sua significatività
- Ad esempio un IMPIEGATO può essere ulteriormente categorizzato in
  - SEGRETARIO, INGEGNERE, TECNICO (tipo di IMPIEGATO)
  - MANAGER (ruolo dell'IMPIEGATO)
  - STIPENDIATO, PAGATO_A_ORE (in base al tipo di contratto)

Figure 4.1
EER diagram notation to represent subclasses and specialization.



Three specializations of EMPLOYEE:
{SECRETARY, TECHNICIAN, ENGINEER}
{MANAGER}
{HOURLY_EMPLOYEE, SALARIED_EMPLOYEE}

SUPERCLASSI E SOTTOCLASSI (1)

- Ognuna delle entità che appartiene a uno dei sottoinsiemi della classe IMPIEGATO è chiamata SOTTOCLASSE
- IMPIEGATO è la SUPERCLASSE per ognuna di queste entità
- Si avrà quindi una relazione SUPERCLASSE/SOTTOCLASSE
  - IMPIEGATO/TECNICO
  - IMPIEGATO/SEGRETARIO
  - IMPIEGATO/MANAGER
  - …
- Sono chiamate anche relazioni IS-A poiché:
  - Un TECNICO è un (IS-A) IMPIEGATO
  - Un MANAGER è un (IS-A) IMPIEGATO
  - …



SUPERCLASSI E SOTTOCLASSI (2)

- Ogni istanza di ognuna delle entità membro di una SOTTOCLASSE rappresenta la stessa istanza dell'entità SUPERCLASSE
- Non può esistere un'istanza di una SOTTOCLASSE che non sia anche istanza della SUPERCLASSE.
- Un'istanza della SUPERCLASSE può OPZIONALMENTE essere inclusa come istanza della SOTTOCLASSE
  - Ogni istanza membro di una sottoclasse EREDITA tutti gli attributo e tutte le relazioni delle istanze della superclasse.

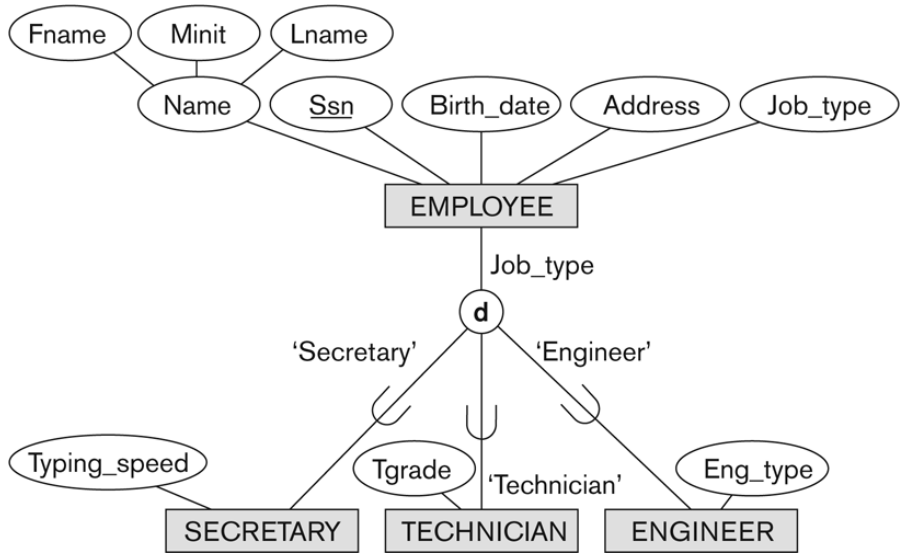


Figure 4.4
EER diagram notation for an attribute-defined specialization on Job_type.

SPECIALIZZAZIONE E GENERALIZZAZIONE

- Il processo di SPECIALIZZAZIONE permette di definire un insieme di sottoclassi per una determinata entità/classe
- La GENERALIZZAZIONE rappresenta il processo inverso alla SPECIALIZZAZIONE

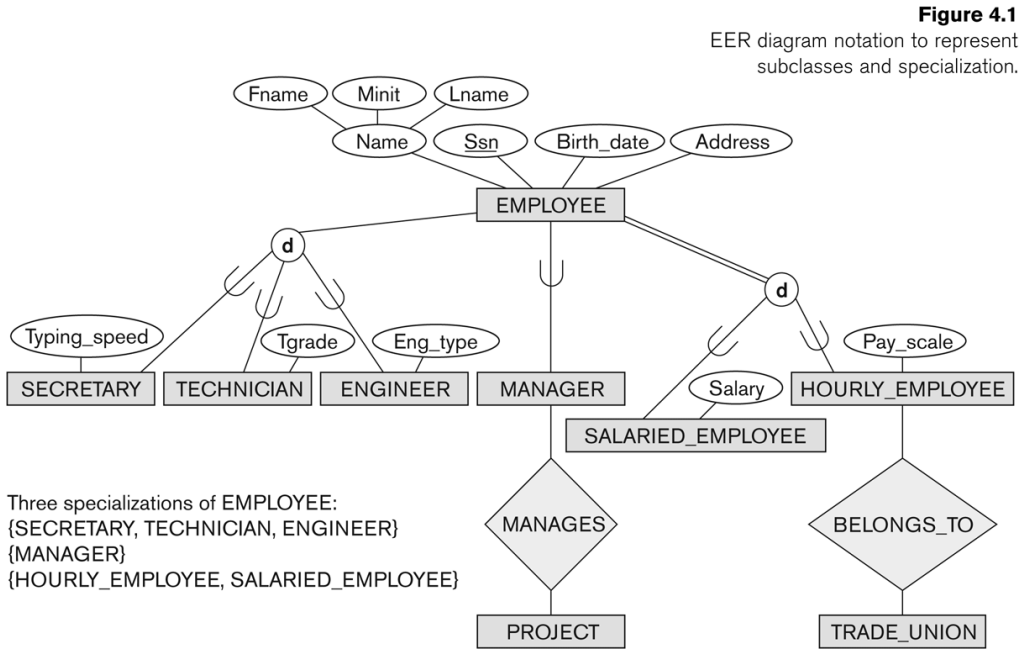


Figure 4.1
EER diagram notation to represent subclasses and specialization.

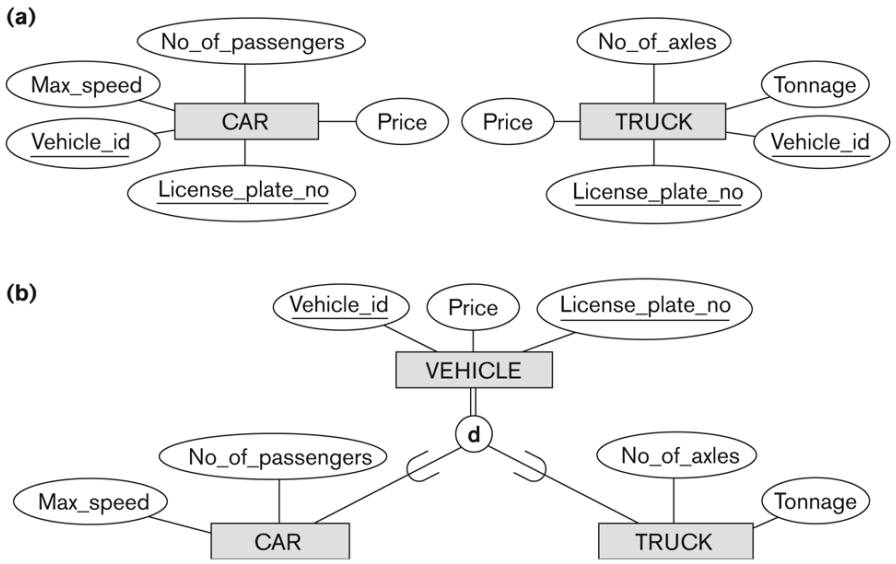


Figure 4.3
Generalization. (a) Two entity types, CAR and TRUCK.
(b) Generalizing CAR and TRUCK into the superclass VEHICLE.

VINCOLI A SPECIALIZZAZIONI/GENERALIZZAZIONI

- Ci sono due vincoli base alle specializzazioni/generalizzazioni
  - Vincolo di Disgiunzione
  - Vincolo di Completezza



VINCOLO DI DISGIUNZIONE

- Specifica che le sottoclassi della specializzazione devono essere **DISGIUNTE**
- Un'istanza può essere membro di al più una sottoclasse
  - Specificata da una ***d*** nel diagramma EER

- Se non sono disgiunte allora la specializzazione è **OVERLAPPING**
- Un'istanza può essere membro di più di una sottoclasse
  - Specificata da una ***o*** nel diagramma EER



VINCOLO DI COMPLETEZZA

- Se è **TOTALE** specifica che ogni istanza nella superclasse deve essere membro di qualche sottoclasse nella specializzazione
- Indicata con una **doppia linea** nel diagramma EER

- Se è **PARZIALE** specifica che possono esserci istanze della superclasse che non appartengono ad alcuna sottoclasse nella specializzazione
- Indicata con una **linea singola** nel diagramma EER

POSSIBILI VINCOLI

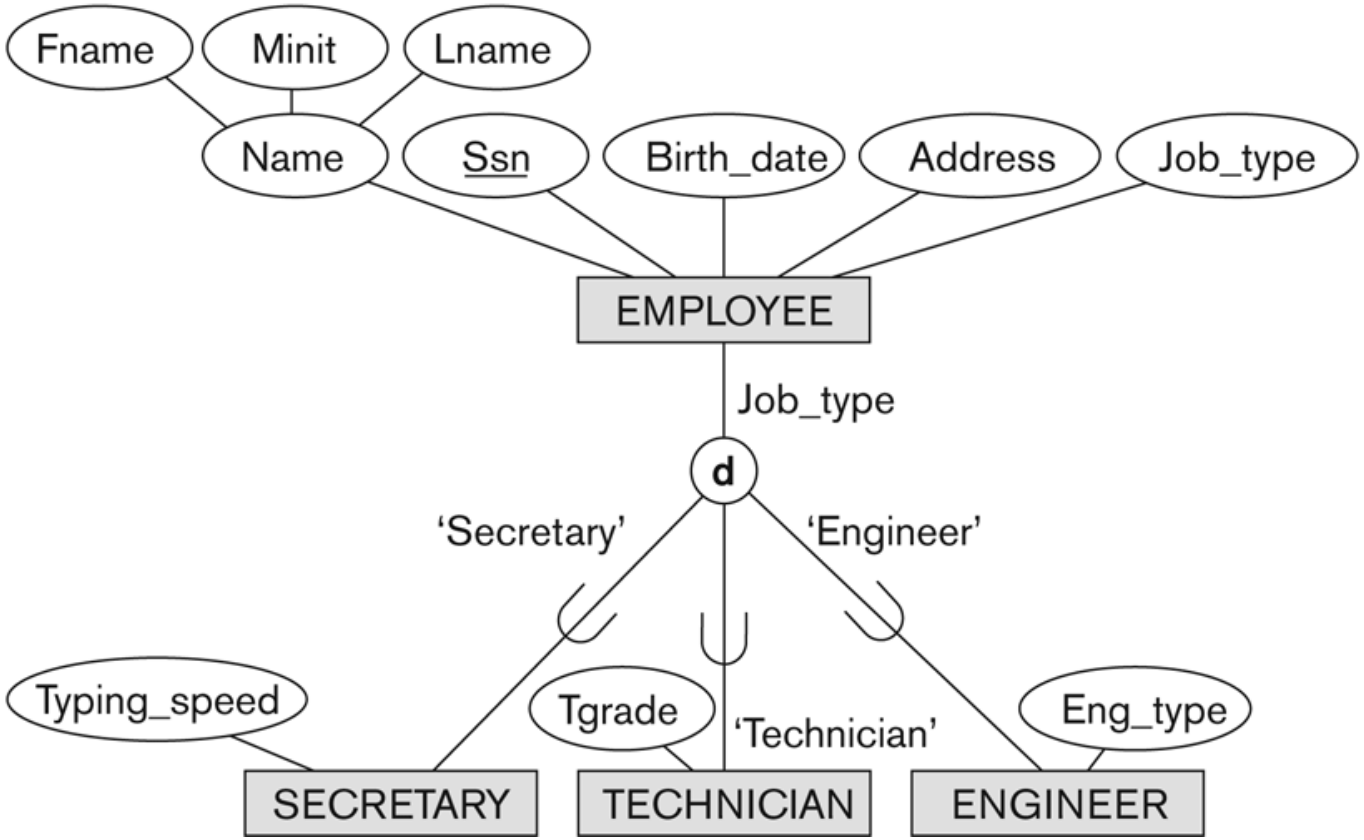
- Possiamo avere quindi quattro tipi di specializzazione/generalizzazione
  - **Disgiunta, Totale**
  - **Disgiunta, Parziale**
  - **Overlapping, Totale**
  - **Overlapping, Parziale**

- Spesso una generalizzazione è totale in quanto la superclasse è derivata dalle sottoclassi.



DISGIUNTA, PARZIALE

Figure 4.4
EER diagram notation for an attribute-defined specialization on Job_type.



OVERLAPPING, TOTALE

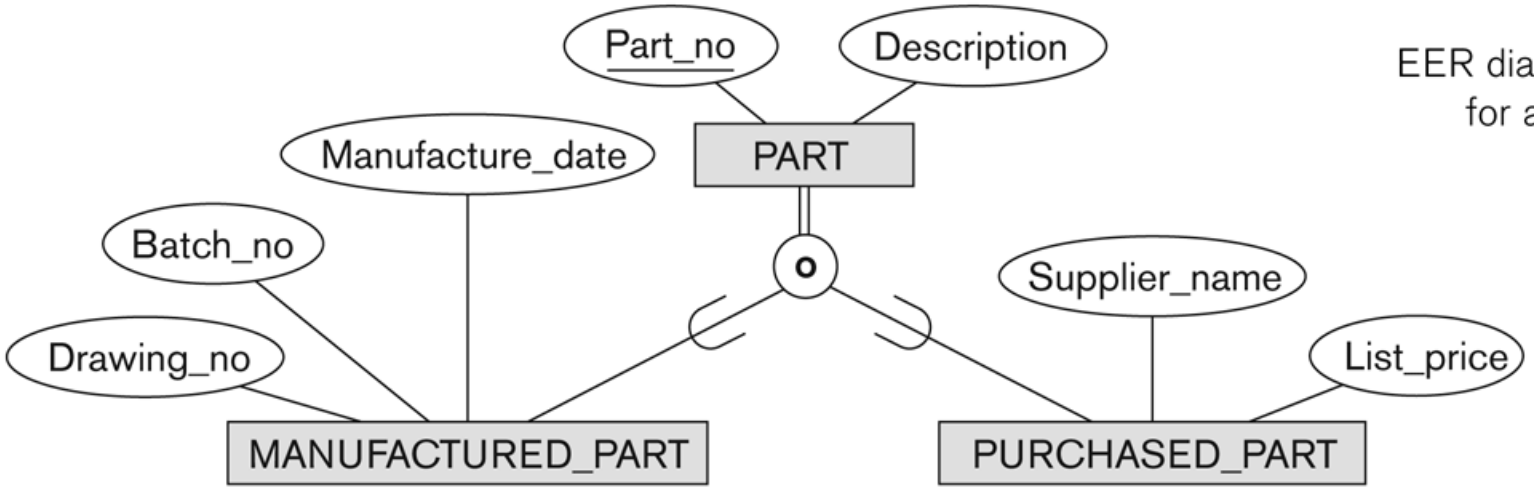
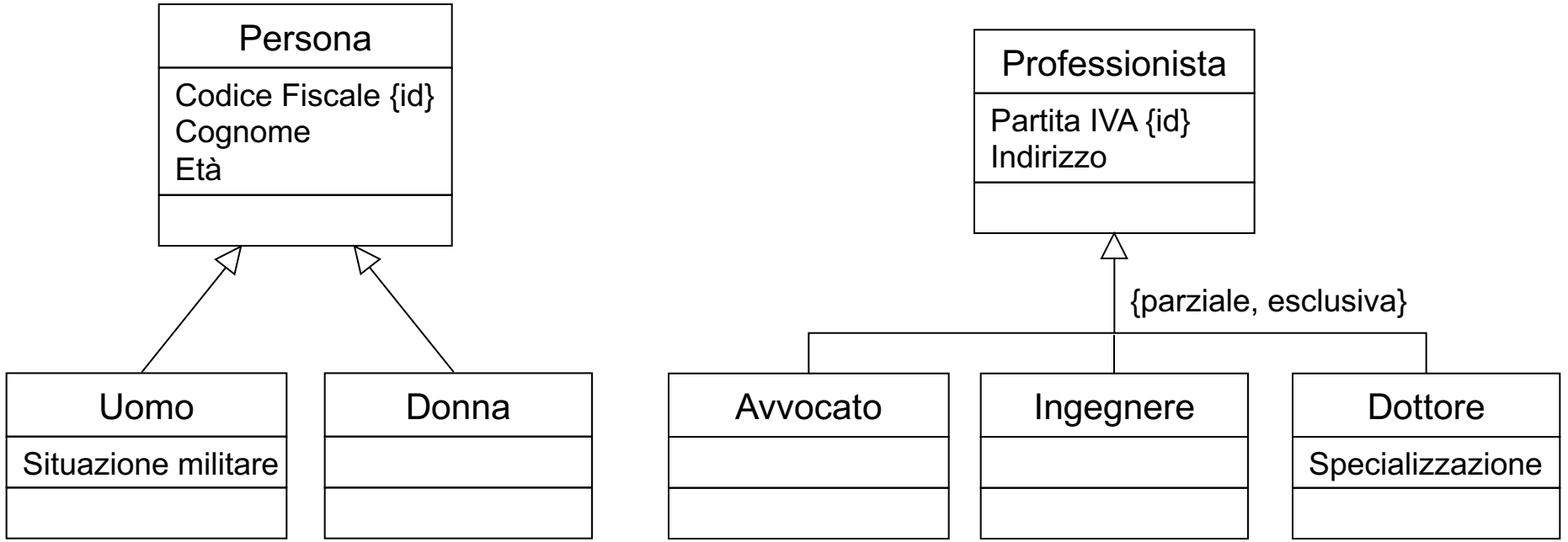


Figure 4.5
EER diagram notation for an overlapping (nondisjoint) specialization.

GENERALIZZAZIONI E SPECIALIZZAZIONI IN UML

- Le generalizzazioni sono molto simili a quelle del modello ER.
- Eventuali proprietà possono essere rappresentate con vincoli racchiusi tra le parentesi { e }.



INFO SULLA LEZIONE

- Tutto il capitolo 3
- Capitolo 4
  - 4.1, 4.2, 4.3

- Il resto del capitolo 4 parla di altri costrutti dell'EER, come l'Unione, che non utilizzeremo particolarmente, ma che può essere comunque interessante imparare.



FINE

Per eventuali domande: (in ordine di preferenza personale)

- Ora.
- Chat di Teams
- Mail: silvio.barra@unina.it